Laporan Tugas Besar 1 IF3170 Inteligensi Artifisial

Semester 5 tahun 2026/2027



Tandborste

Fahd Muhammad Zahid	13524078
Al Farabi	13524086
Ishaq Irfan Farizal	13524094

DAFTAR ISI

DAFTAR ISI **2**
Bab I : Deskripsi Persoalan **4**
1.1 Persoalan dan Tujuan Optimisasi 4
Bab II : Pembahasan Implementasi **7**
2.1 Representasi State, Inisialisasi, dan Operator Tetangga 7
2.1.1 Model Paket dan Truk 7
2.2 Fungsi Objektif dan Penanganan Kendala 9
2.2.1 Fungsi Objektif Utama 9
2.3 Hill-Climbing 11
2.3.1 Steepest Ascent Hill-Climbing 12
2.4 Simulated Annealing 15
2.4.1 Kriteria Penerimaan Metropolis dan Fluktuasi Probabilitas 15
2.4.2 Penurunan Suhu Geometrik dan Parameter Algoritma 16
2.4.3 Solusi Akhir dan Pencatatan Metrik 16
2.4.4 Kode Sumber Implementasi Simulated Annealing 17
2.5 Genetic Algorithm 21
2.5.1 Representasi Kromosom dan Gen 21
2.5.2 Heuristik Decoding Bottom-Left-Back (BLB) 22
2.5.3 Operator Genetika dan Elitisme 22
2.5.4 Kode Sumber Implementasi Genetic Algorithm 22
2.6 Visualisasi State dan Hasil Eksperimen 31
2.6.1 Gambaran Umum Visualisasi 31
2.6.2 Variabel Permasalahan 31
2.6.3 Representasi Solusi dan Batasan Permasalahan 32
2.6.4 Visualisasi Grafik Hasil Pencarian 33
2.6.5 Integrasi dan Penyimpanan Hasil Visualisasi 34
2.6.6 Integrasi dan Penyimpanan Hasil Visualisasi 35
2.7 Fungsi Objektif Alternatif 36
2.7.1 Formulasi Matematis Fungsi Objektif Alternatif 36
2.7.2 Rasionalisasi Komponen Objektif 36
2.7.3 Integrasi dan Pemanggilan pada CLI 37
2.7.4 Kode Sumber Objektif Alternatif 37
Bab III : Hasil dan Analisis Eksperimen **40**
3.1 Konfigurasi dan Skenario Eksperimen 40
3.2 Hasil dan Analisis Eksperimen Hill-Climbing 41
3.2.1 Metrik Khusus dan Interpretasi Penghitung 42
3.3 Hasil dan Analisis Eksperimen Simulated Annealing 46
3.3.1 Karakteristik Kasus Uji yang digunakan 46
3.3.2 Tabel Rekapitulasi Hasil Eksperimen SA 47
3.3.3 Bukti Visualisasi State dan Grafik Metrik SA 47

3.3.4 Analisis Karakteristik dan Perilaku Pencarian SA....48
3.4 Hasil dan Analisis Eksperimen Genetic Algorithm....49
3.4.1 Studi Parameter 1: Variasi Jumlah Generasi (Populasi Tetap = 30)....49
3.4.2 Studi Parameter 2: Variasi Ukuran Populasi (Generasi Tetap = 50)....50
3.4.3 Bukti Visualisasi State dan Grafik Metrik GA....51
3.4.4 Analisis Hasil Eksperimen dan Visualisasi GA....52
3.5 Analisis dan Perbandingan Antaralgoritma....53
3.5.1 Kualitas Solusi terhadap Batas Atas....53
Bab IV : Kesimpulan dan Saran....55
4.1 Kesimpulan....55
4.2 Saran....55
SURAT PERNYATAAN PENGGUNAAN AI....56
Lampiran....58
A. Pranala Github....58
B. Tabel Ceklis Spesifikasi....58
C. Tabel Pembagian Tugas....59

Bab I : Deskripsi Persoalan

1.1 Persoalan dan Tujuan Optimisasi

Mejiro Express merupakan sebuah perusahaan logistik darat yang sedang melakukan uji coba pengiriman barang menggunakan truk peti kemas. Dalam proses pengiriman, perusahaan perlu menentukan konfigurasi penempatan paket agar nilai total barang yang diangkut dapat dimaksimalkan. Permasalahan ini dikenal sebagai 3D Container Loading Problem, yaitu permasalahan optimasi penempatan sejumlah paket berbentuk tiga dimensi ke dalam ruang peti kemas yang memiliki ukuran dan kapasitas angkut terbatas.

Setiap paket memiliki karakteristik yang berbeda, seperti dimensi, berat, nilai barang, tingkat kerapuhan, dan estimasi waktu kedatangan (Estimated Time of Arrival atau ETA). Perbedaan karakteristik tersebut menyebabkan tidak semua paket dapat dimasukkan secara bersamaan. Selain mempertimbangkan nilai barang, konfigurasi penempatan juga harus memenuhi batasan fisik, seperti kapasitas angkut maksimum, batas ruang peti kemas, dan ketentuan penumpukan paket.

Pada tugas besar ini, dikembangkan sebuah program berbasis Python yang menerapkan algoritma local search untuk mencari konfigurasi pemuatan paket dengan nilai fungsi objektif setinggi mungkin. Algoritma yang digunakan meliputi beberapa varian Hill-Climbing, Simulated Annealing, dan Genetic Algorithm. Program juga menyediakan visualisasi tiga dimensi untuk memperlihatkan konfigurasi paket serta grafik hasil pencarian untuk membantu proses analisis dan perbandingan algoritma.

1.2 Variable Permasalahan

Permasalahan direpresentasikan menggunakan dua entitas utama, yaitu Package dan Truck. Entitas Package digunakan untuk menyimpan karakteristik dan status penempatan setiap paket, sedangkan Truck merepresentasikan ruang peti kemas beserta kapasitas maksimum yang tersedia.

1.2.1 Atribut Entitas Package dan Truck

Entitas	Atribut	Keterangan
Package	`id`	Identitas unik setiap paket.
Package	`w, l, h`	Dimensi awal paket berupa lebar, panjang, dan tinggi.
Package	`value`	Nilai barang yang digunakan dalam perhitungan fungsi objektif.

Package	`weight`	Berat paket dalam satuan unit berat.
Package	`is_fragile`	Status barang pecah-belah yang menentukan ketentuan penumpukan.
Package	`eta`	Estimasi waktu kedatangan paket dalam satuan hari.
Package	`orientation_idx`	Indeks orientasi paket yang menentukan dimensi setelah rotasi.
Package	`x, y, z`	Koordinat posisi paket di dalam peti kemas.
Package	`is_loaded`	Status yang menunjukkan apakah paket dimuat ke dalam peti kemas.
Truck	`id`	Identitas unik truk.
Truck	`w, l, h`	Dimensi ruang peti kemas.
Truck	`max_capacity`	Total berat maksimum yang dapat diangkut truk.

Tabel 1.1 Atribut package dan penjelasan

Dalam implementasi, seluruh atribut tersebut didefinisikan melalui kelas `Package` dan `Truck` pada berkas `src/core/models.py`. Setiap konfigurasi pemuatan kemudian direpresentasikan sebagai objek `State` yang menyimpan informasi truk, seluruh paket, serta status penempatannya.

1.3 Representasi Solusi dan Batasan Permasalahan

Representasi solusi menggunakan koordinat tiga dimensi (`x, y, z`) dengan nilai bilangan bulat. Sumbu `x` menunjukkan lebar, sumbu `y` menunjukkan panjang, dan sumbu `z` menunjukkan tinggi peti kemas. Setiap paket dapat ditempatkan dalam enam kemungkinan orientasi yang diperoleh melalui perubahan susunan dimensi akibat rotasi 90° terhadap sumbu koordinat.

Tidak seluruh paket harus dimasukkan ke dalam peti kemas. Paket yang tidak dimuat direpresentasikan dengan status `is_loaded = False`, sedangkan paket yang berada di dalam peti kemas memiliki status `is_loaded = True` beserta informasi koordinat dan orientasinya.

Untuk menghasilkan konfigurasi yang valid, terdapat beberapa batasan yang harus dipenuhi:

1) **Batas ruang peti kemas**. Seluruh bagian paket yang dimuat harus berada di dalam peti kemas dan tidak boleh menembus batas ruang yang tersedia.

2) **Tabrakan antarpaket**. Dua paket tidak diperbolehkan menempati volume ruang yang saling bertumpang tindih.

3) **Tumpuan paket**. Paket tidak diperbolehkan melayang. Paket yang tidak berada di lantai peti kemas harus memiliki setidaknya satu bagian alas yang ditopang oleh permukaan atas paket lain.

4) **Barang pecah-belah**. Paket dengan atribut `is_fragile = True` tidak diperbolehkan menjadi tumpuan bagi paket lainnya.

5) **Kapasitas angkut**. Jumlah berat seluruh paket yang dimuat tidak boleh melebihi nilai `max_capacity` milik truk.

Konfigurasi awal (*initial state*) dibangkitkan secara acak dengan tetap mempertimbangkan seluruh batasan tersebut. Selanjutnya, algoritma pencarian dapat menghasilkan konfigurasi tetangga melalui operasi *move*, *swap*, dan *rotate* untuk mengeksplorasi kemungkinan penempatan yang berbeda.

1.4 Tujuan dan Cakupan Penyelesaian

Tujuan utama penyelesaian masalah adalah memaksimalkan jumlah nilai barang yang berhasil dimuat ke dalam satu peti kemas. Fungsi objektif utama didefinisikan sebagai penjumlahan atribut value dari seluruh paket yang memiliki status `is_loaded = True`, dengan tetap memenuhi seluruh batasan fisik yang telah ditentukan.

Program mengimplementasikan empat varian *Hill-Climbing, yaitu Steepest Ascent Hill-Climbing, Hill-Climbing with Sideways Move, Stochastic Hill-Climbing, dan Random Restart Hill-Climbing*, serta algoritma *Simulated Annealing* dan *Genetic Algorithm*. Sebagai fitur tambahan, program juga menyediakan fungsi objektif alternatif yang mempertimbangkan urgensi pengiriman berdasarkan ETA dan tingkat pemanfaatan volume peti kemas.

Hasil pencarian disajikan melalui visualisasi konfigurasi awal dan akhir dalam bentuk tiga dimensi, grafik perkembangan nilai fungsi objektif, serta metrik yang berkaitan dengan proses pencarian. Seluruh hasil tersebut digunakan sebagai dasar untuk mengevaluasi kualitas solusi, durasi pencarian, dan karakteristik masing-masing algoritma.

Cakupan implementasi dibatasi pada optimasi pemuatan satu truk peti kemas. Optimasi pengiriman yang melibatkan beberapa truk secara bersamaan tidak termasuk dalam implementasi program ini.

Bab II : Pembahasan Implementasi

2.1 Representasi State, Inisialisasi, dan Operator Tetangga

2.1.1 Model Paket dan Truk

Kami merepresentasikan persoalan melalui kelas Package dan Truck pada berkas src/core/models.py. Package menyimpan atribut dasar paket serta status pemuatan, koordinat (x, y, z), dan orientation_idx. Truck menyimpan dimensi ruang peti kemas dan max_capacity dalam satuan unit berat. Pemeriksaan penempatan memakai dimensi efektif hasil orientasi agar perubahan bentuk akibat rotasi ikut diperhitungkan.

Enam orientasi direpresentasikan sebagai permutasi dimensi (w, l, h), yaitu (w, l, h), (w, h, l), (l, w, h), (l, h, w), (h, w, l), dan (h, l, w). Indeks 0 sampai 5 menunjuk ke ORIENTATIONS. Paket dengan dimensi yang sama pada beberapa sumbu dapat mempunyai bentuk efektif yang identik pada lebih dari satu indeks. Fungsi neighbor_rotate mencoba orientasi lain dari himpunan ini; implementasi tidak membatasi setiap transisi ke tepat satu rotasi elementer sebesar 90 derajat.

2.1.2 Representasi State dan Inisialisasi Acak

Objek State menyimpan satu truk, seluruh paket, serta fungsi objektif yang digunakan. Properti loaded_packages dan unloaded_packages membagi paket berdasarkan is_loaded. Paket di luar truk tetap tercatat dalam state, tetapi posisi yang diekspor bernilai null dan paket tersebut tidak berkontribusi terhadap muatan maupun skor. Keadaan ini berbeda dari balok yang sebagian menembus dinding truk, yang selalu dinilai tidak valid.

Inisialisasi dilakukan oleh State.create_random_state(). Seluruh paket disalin dan mula-mula ditempatkan di luar truk dengan orientasi acak. Urutan paket kemudian diacak. Untuk setiap paket, terdapat peluang 0,8 untuk mencoba pemuatan. Titik kandidat dan urutan enam orientasi diacak, lalu penempatan pertama yang memenuhi state.is_valid() dipertahankan. Jika seluruh percobaan gagal, paket tetap berada di luar. Dengan demikian, hasil awal dapat berbeda pada setiap seed tanpa memasukkan konfigurasi yang melanggar kendala.

State.clone() menyalin setiap objek Package beserta atribut penempatannya dan mempertahankan referensi Truck serta fungsi objektif. Truk tidak dimodifikasi oleh operator tetangga. Setiap perubahan kandidat dilakukan pada salinan state sehingga kandidat yang gagal tidak menimpa konfigurasi asal.

Potongan kode penyalinan state (src/core/state.py, baris 59–66).

```
    def clone(self) -> "State":
        """Membuat salinan mendalam (deep copy) dari state."""
        cloned_packages = [p.clone() for p in self.packages]
        return State(
```

```
            truck=self.truck,
            packages=cloned_packages,
            objective_fn=self.objective_fn,
            )
```

2.1.3 Heuristik Titik Kandidat dan Operator Tetangga

Kami menggunakan _get_candidate_anchors() untuk membatasi koordinat yang dicoba agar pembangkitan tetangga tidak perlu memeriksa seluruh koordinat bilangan bulat. Titiknya mencakup (0, 0, 0), sisi kanan dan belakang paket di lantai, serta permukaan atas paket yang tidak fragile. Tiga titik acak di lantai ditambahkan untuk memberi variasi. Titik di luar batas awal disaring saat pembentukan anchor; kecocokan seluruh dimensi paket tetap diperiksa melalui validasi. Heuristik ini mengurangi kandidat yang diperiksa, tetapi dapat melewatkan penempatan valid di koordinat lain.

Operator	Perubahan pada kandidat	Pemeriksaan
neighbor_move	Memuat paket luar, memindahkan paket dalam, atau membongkar satu paket.	Kandidat hanya dikembalikan jika seluruh state valid; membongkar penopang dapat ditolak.
neighbor_swap	Menukar koordinat dua paket dalam; atau mengganti paket dalam dengan paket luar pada posisi yang sama.	Kasus dalam–luar mencoba enam orientasi acak; dua paket luar tidak menghasilkan kandidat.
neighbor_rotate	Memilih satu paket dalam dan mencoba indeks orientasi selain indeks aktif.	Posisi dipertahankan dan seluruh kendala diperiksa kembali.

Tabel 2.1 Operator pembangkitan tetangga pada representasi State.

get_random_neighbor() memilih operator move, swap, atau rotate secara acak, kemudian mengulang sampai menemukan kandidat valid atau mencapai max_tries (default 25). get_all_neighbors() mengumpulkan sampel unik, dengan paling banyak dua kali sample_limit upaya pemanggilan get_random_neighbor(max_tries=5). Identitas sampel mencakup ID, status loaded, posisi, dan orientasi seluruh paket. Jumlah kandidat dapat lebih kecil dari sample_limit karena percobaan gagal atau menghasilkan konfigurasi yang sama. Oleh sebab itu, nama get_all_neighbors tidak berarti enumerasi lengkap lingkungan.

Potongan kode pemilihan tetangga acak (src/core/state.py, baris 299–305).

```
        operators = [neighbor_move, neighbor_swap, neighbor_rotate]
         for _ in range(max_tries):
```

```
            op = random.choice(operators)
            neighbor = op(state)
            if neighbor is not None and neighbor.is_valid():
            return neighbor
        return None
```

Kelas atau Fungsi	Masukan utama	Keluaran dan kegunaan
Package.current_dims()	Dimensi dan orientation_idx	Tuple dimensi efektif untuk geometri balok.
Package.clone()	Atribut dan status paket	Objek Package baru dengan konfigurasi yang sama.
State.create_random_state()	Truck, daftar Package, objective_fn	State awal acak yang valid.
State.clone()	State asal	State baru dengan salinan seluruh paket.
_get_candidate_anchors()	State aktif	Daftar koordinat kandidat penempatan.
get_random_neighbor()	State, max_tries	Satu tetangga valid atau None.
get_all_neighbors()	State, sample_limit	Daftar sampel tetangga valid tanpa duplikasi.

Tabel 2.2 Kelas dan fungsi utama pada representasi state.

Potongan kode inti dicantumkan pada subbab yang membahas fungsinya. Penjelasan di atas mengikuti perilaku program yang dipakai dalam eksperimen, termasuk batas sampling dan keterbatasan operator rotasi.

2.2 Fungsi Objektif dan Penanganan Kendala

2.2.1 Fungsi Objektif Utama

Kami menggunakan default_objective() untuk menjumlahkan nilai seluruh paket yang dimuat, sesuai tujuan utama persoalan. Dengan L(s) menyatakan himpunan paket berstatus is_loaded = True pada state s, fungsi objektifnya adalah $f(s) = \Sigma_{i \in} L(s)\ value_i$. Atribut ETA dan utilisasi volume digunakan pada fungsi alternatif di Subbab 2.7.

Potongan kode objektif utama (src/core/objectives.py, baris 7–12).

```
def default_objective(state: "State") -> float:
	"""
	Fungsi objektif default (Spesifikasi Bagian Objective
Function):
	Memaksimalkan total value dari paket-paket yang berhasil
dimuat ke dalam truk.
	"""
	return sum(p.value for p in state.loaded_packages)
```

Evaluasi dilakukan melalui State.get_score(). Jika state.is_valid() bernilai False, skor dikembalikan sebagai 0,0; jika valid, nilai objective_fn(state) dikembalikan sebagai float. State kosong yang valid juga memiliki skor 0,0. Karena itu, nilai nol saja tidak cukup untuk menentukan validitas; properti is_valid dan pesan get_errors() tetap diperlukan.

Potongan kode evaluasi skor state (src/core/state.py, baris 49–51)./

```
if not self.is_valid():
	return 0.0
 return float(self.objective_fn(self))
```

2.2.2 Pemeriksaan Kendala Fisik

Fungsi	**Syarat yang diperiksa**
is_within_bounds()	Untuk dimensi efektif (w_i, l_i, h_i): x_i, y_i, $z_i \geq 0$; $x_i + w_i \leq W$; $y_i + l_i \leq L$; $z_i + h_i \leq H$.
is_overlapping_3d()	Dua balok bertabrakan jika intervalnya beririsan secara ketat pada ketiga sumbu. Sentuhan permukaan saja tidak dianggap overlap.
is_supported()	Paket berada di lantai ($z_i = 0$), atau ada paket j dengan $z\square + h\square = z_i$ serta irisan proyeksi x–y yang positif. Minimal satu titik alas bertumpu; seluruh alas tidak harus ditopang.
is_fragile_violated()	Pelanggaran terjadi jika permukaan atas paket fragile bersentuhan dengan alas paket lain dan proyeksi x–y keduanya beririsan.
is_capacity_exceeded()	Total weight seluruh paket termuat harus ≤ max_capacity; nilai yang sama dengan kapasitas masih valid.

check_all_constraints()	Memeriksa kapasitas, dinding, seluruh pasangan balok, tumpuan, dan fragile; mengembalikan pasangan (valid, daftar pesan kesalahan).

Tabel 2.3 Pemeriksaan kendala pada konfigurasi muatan.

Koordinat dan dimensi berbentuk bilangan bulat. Oleh karena itu, irisan proyeksi positif pada ketinggian yang sama memastikan adanya titik alas yang bertumpu sesuai ketentuan tugas. Pemeriksaan tumpuan tidak mengharuskan seluruh bidang alas tertutup dan tidak menerapkan model mekanika seperti keseimbangan pusat massa. Barang fragile tetap diperiksa terpisah sehingga dukungan oleh barang fragile tidak membuat konfigurasi diterima.

Potongan kode pemeriksaan tabrakan tiga dimensi (src/core/constraints.py, baris 23–34).

```
w1, l1, h1 = pkg1.current_dims()
 w2, l2, h2 = pkg2.current_dims()
 if pkg1.x + w1 <= pkg2.x or pkg2.x + w2 <= pkg1.x:
       return False
 if pkg1.y + l1 <= pkg2.y or pkg2.y + l2 <= pkg1.y:
       return False
 if pkg1.z + h1 <= pkg2.z or pkg2.z + h2 <= pkg1.z:
       return False
 return True
```

Seluruh pemeriksaan diterapkan terhadap loaded_packages. Operator yang gagal menghasilkan konfigurasi valid mengembalikan None, sedangkan pembangkit tetangga mengulang sesuai anggaran percobaan. Penalti nol pada get_score() menjadi lapisan evaluasi tambahan, bukan satu-satunya mekanisme menjaga validitas.

2.3 Hill-Climbing

Hill-Climbing melakukan perbaikan lokal berdasarkan nilai tetangga dari state saat ini [3]. Kami mengimplementasikan empat varian yang berbeda dalam pemilihan kandidat, penerimaan nilai datar, serta cara memulai ulang pencarian. Seluruh varian menggunakan representasi State dan fungsi objektif yang sama.

Varian	**Pemilihan dan penerimaan**	**Terminasi**
Steepest Ascent	Memilih skor maksimum dari sampel unik; menerima hanya jika skor meningkat.	Sampel kosong, kandidat terbaik tidak lebih baik, atau batas iterasi.

Sideways Move	Memilih yang terbaik dari sampel; menerima peningkatan dan skor sama dalam batas sideways.	Sampel kosong, semua kandidat lebih buruk, batas sideways terlampaui, atau batas iterasi.
Stochastic	Mengambil satu kandidat acak per iterasi; menerima peningkatan ketat.	Seluruh max_iterations selesai, walaupun skor tidak lagi membaik.
Random Restart	Menjalankan Steepest dari beberapa state awal acak dan menyimpan hasil terbaik di antara run.	Seluruh max_restarts inner run selesai; tiap inner run mempunyai batas iterasi sendiri.

Tabel 2.4 Perbedaan mekanisme empat varian Hill-Cl

2.3.1 Steepest Ascent Hill-Climbing

Fungsi steepest_ascent_hill_climbing(initial_state, max_iterations=300, sample_neighbors=25) menyalin initial_state dan menyimpan skor awal pada iterasi 0. Setiap iterasi membangkitkan sampel tetangga valid, memilih max(neighbors, key=get_score), lalu berpindah hanya jika skornya lebih besar. Jika skor maksimum sampel kurang dari atau sama dengan skor aktif, skor aktif dicatat sekali lagi dan pencarian dihentikan. Ketiadaan perbaikan hanya berlaku pada sampel yang diperiksa, sehingga hasil akhir tidak membuktikan optimum lokal terhadap seluruh kemungkinan koordinat.

Potongan kode pemilihan langkah Steepest Ascent (src/algorithms/hill_climbing.py, baris 38–48).

```
best_neighbor = max(neighbors, key=lambda s: s.get_score())
 best_score = best_neighbor.get_score()
 current_score = current.get_score()
 if best_score <= current_score:
       score_history.append(current_score)
       iteration_history.append(iteration)
       break
 current = best_neighbor
```

2.3.2 Hill-Climbing with Sideways Move

Fungsi hill_climbing_sideways_move() menggunakan selisih delta = best_score − current_score. Delta positif diterima dan mengatur ulang penghitung sideways berturut-turut; delta negatif menghentikan pencarian. Pada delta nol, consecutive_sideways dan total_sideways_count dinaikkan terlebih dahulu. Transisi kemudian hanya diterima apabila consecutive_sideways tidak melebihi max_sideways. Pada pemanggilan langsung, default batas iterasinya 500, sedangkan CLI selalu memasok

--max-iterations (default 300). Eksperimen ini menggunakan batas 300 agar konfigurasinya eksplisit.

Urutan pembaruan penghitung perlu diperhatikan. Dengan max_sideways = 50, percobaan datar ke-51 pada suatu rentetan menambah penghitung dan menutup pencarian tanpa mengambil transisi itu. Oleh sebab itu, extra_metrics[total_sideways_performed] merupakan jumlah percobaan datar tercatat, yang dapat lebih besar satu daripada sideways yang benar-benar diterima. Nilai mentah, jumlah diterima, dan rentetan terakhir dilaporkan terpisah pada Subbab 3.2.

Potongan kode penerimaan langkah Sideways (src/algorithms/hill_climbing.py, baris 103–120).

```
if delta < 0:
      score_history.append(current_score)
      iteration_history.append(iteration)
      break
 elif delta == 0:
      consecutive_sideways += 1
      total_sideways_count += 1
      if consecutive_sideways > max_sideways:
        score_history.append(current_score)
        iteration_history.append(iteration)
      break
      current = best_neighbor
 else:
      consecutive_sideways = 0
      current = best_neighbor
```

2.3.3 Stochastic Hill-Climbing

Fungsi stochastic_hill_climbing(initial_state, max_iterations=300) memanggil get_random_neighbor(max_tries=10) sekali pada setiap iterasi. Kandidat yang ada diterima hanya jika skornya lebih tinggi. Kandidat datar, kandidat lebih buruk, maupun None tidak mengubah current. Riwayat tetap ditambah pada setiap iterasi. Tidak terdapat terminasi dini karena plateau, sehingga kurva dapat mendatar panjang hingga anggaran iterasi habis. Metrik accepted_moves_count menghitung peningkatan yang benar-benar diterima.

Potongan kode iterasi Stochastic Hill-Climbing (src/algorithms/hill_climbing.py, baris 164–173).

```
for iteration in range(1, max_iterations + 1):
      neighbor = get_random_neighbor(current, max_tries=10)
      if neighbor is not None:
      if neighbor.get_score() > current.get_score():
            current = neighbor
```

```
                accepted_moves += 1
            score_history.append(current.get_score())
              iteration_history.append(iteration)
```

2.3.4 Random Restart Hill-Climbing

Fungsi random_restart_hill_climbing(truck, packages, max_restarts=5, iterations_per_restart=100) membangkitkan state awal baru pada setiap inner run. Pencarian dalamnya menggunakan Steepest dengan sample_neighbors=15 yang ditetapkan langsung pada kode; --sample-neighbors tidak mengubah nilai tersebut. Program melakukan tepat max_restarts inner run, sehingga parameter 5 berarti lima pencarian, termasuk pencarian pertama, bukan satu pencarian awal ditambah lima pencarian tambahan.

Potongan kode pencarian pada setiap restart (src/algorithms/hill_climbing.py, baris 219–230).

```
for r in range(max_restarts):
       init_state = State.create_random_state(truck, packages,
objective_fn)
       if first_initial_state is None:
       first_initial_state = init_state.clone()
       run_res = steepest_ascent_hill_climbing(
       init_state,
         max_iterations=iterations_per_restart,
       sample_neighbors=15,
       )
```

initial_state pada hasil gabungan adalah state awal inner run pertama, sedangkan final_state adalah solusi terbaik di antara seluruh inner run. scores_per_restart dan iterations_per_restart_actual menyimpan hasil masing-masing pencarian. Riwayat digabung dengan menyertakan titik awal setiap inner run. Akibatnya, result.iterations menghitung entri riwayat gabungan, bukan penjumlahan iterasi pencarian dalam. Kurva gabungan boleh turun pada batas restart dan sumbu horizontalnya harus dibaca sebagai urutan rekaman, bukan lintasan perbaikan tunggal.

Potongan kode penyimpanan hasil restart terbaik (src/algorithms/hill_climbing.py, baris 241–243).

```
if best_global_state is None or run_res.final_score >
best_global_score:
       best_global_score = run_res.final_score
       best_global_state = run_res.final_state.clone()
```

2.3.5 Pengukuran dan Integrasi Pencarian

Keempat fungsi mengembalikan SearchResult yang memuat state awal–akhir, skor, iterations, duration_seconds, score_history, iteration_history, dan extra_metrics. Durasi diukur di dalam fungsi algoritma dengan time.time(). Untuk Steepest, Sideways, dan Stochastic, pembangkitan initial_state oleh run_single_search() berada di luar pengukuran; pada Random Restart, pembangkitan awal setiap inner run termasuk dalam durasi. Semua batas waktu pada hasil eksperimen mengikuti definisi program ini.

2.4 Simulated Annealing

Simulated Annealing (SA) merupakan algoritma optimasi lokal berbasis probabilistik yang mengadaptasi fenomena termodinamika pada proses pendinginan logam (*annealing)* [1], [2]. Pada proses metalurgi, material dipanaskan hingga temperatur tinggi lalu didinginkan secara bertahap agar partikel-partikel atomnya dapat menyusun diri ke dalam konfigurasi kisi kristal dengan energi minimum [1]. Dalam ranah kecerdasan artifisial dan optimasi kombinatorial, analogi ini dimanfaatkan untuk mengatasi kelemahan utama algoritma *Hill-Climbing* yang kerap terperangkap pada puncak lokal (*local optima*).

2.4.1 Kriteria Penerimaan Metropolis dan Fluktuasi Probabilitas

Pada setiap iterasi, algoritma Simulated Annealing membangkitkan satu tetangga acak *s'* dari state saat ini *s*. Evaluasi selisih nilai fungsi objektif dinyatakan sebagai [1], [6] :

$$\Delta E \; = \; skor(s') \; - \; skor(s)$$

Kriteria penerimaan kandidat tetangga diatur mengikuti aturan Metropolis [2]:

1. **Solus Membaik (**$\Delta E > 0$): Kandidat *s'* selalu diterima secara deterministik ($s \leftarrow s'$).
2. **Solusi Memburuk atau Datar** ($\Delta E < 0$): Transisi ke solusi yang lebih buruk tidak langsung ditolak, melainkan diterima dengan probabilitas penerimaan [1]:

$$P(terima) \; = \; exp(\frac{\Delta E}{T}) \; - \; skor(-\frac{|\Delta E|}{T})$$

Di mana sebuah bilangan acak $r \sim U(0,1)$ dibangkitkan. Apabila $r < P(terima)$, solusi yang memburuk tersebut tetap diambil sebagai state aktif.

Sebagaimana dipaparkan oleh GeeksforGeeks [1], pada temperatur *T* tinggi (tahap awal), nilai $P(terima)$ mendekati 1 sehingga algoritma leluasa mengeksplorasi ruang pencarian (*broad exploration*). Ketika temperatur meluruh mendekati nol, probabilitas penerimaan solusi buruk mengecil drastis, mengarahkan algoritma menjadi eksploitatif murni (*fine-tuning exploitation*) [1], [6].

2.4.2 Penurunan Suhu Geometrik dan Parameter Algoritma

Penurunan temperatur dikendalikan oleh skema pendinginan geometrik (*geometric cooling schedule*) [1]:

$$T_{k+1} = \alpha . T_k$$

Dengan parameter yang diterapkan :

- T_0: Temperatur awal pencarian (*default* : 200.0) [1].
- α : Koefisien laju penurunan suhu, di mana $0 < \alpha < 1$ (default: 0.95). Laju pendinginan yang terlalu cepat berisiko memicu konvergensi prematur (*premature convergence*) [1].
- T_{min} : Ambang temperatur terendah untuk terminasi pencarian (default : 0.01)
- `Iter_per_temp` : Jumlah iterasi pencarian tetangga per tingkat temperatur (default: 10).
- `stuck_limit` : Ambang penolakan berturut-turut untuk mencatat frekuensi jebakan optimum lokal (`stuck_count`) (default: 50).

2.4.3 Solusi Akhir dan Pencatatan Metrik

Implementasi program menetapkan beberapa kaidah penting:

1. ***Best Visited State* :** Riwayat objektif (`score_history`) merekam nilai *state aktif* yang mengalami fluktuasi, namun solusi akhir yang dikembalikan (`final_state`) adalah *state* terbaik global yang pernah dikunjungi (*best-ever visited state)* [6].
2. **Riwayat Probabilitas**: Nilai $exp(\frac{\Delta E}{T})$ dicatat pada `prob_history` beserta nomor iterasinya setiap kali kandidat dengan $\Delta E \leq 0$ dievaluasi [1].
3. **Pencatatan Jebakan Lokal (*Stuck Counter)*:** Jika penolakan tetangga berturut-turut mencapai `stuck_limit` (50 kali), pencacah `stuck_count` bertambah 1 dan perhitungan di-reset.

2.4.4 Kode Sumber Implementasi Simulated Annealing

```
import math

import random

import time

from typing import List


from src.core.state import State, get_random_neighbor

from .base import SearchResult



def simulated_annealing(

    initial_state: State,

    t0: float = 200.0,

    alpha: float = 0.95,

    t_min: float = 0.01,

    iter_per_temp: int = 10,

    stuck_limit: int = 50,

) -> SearchResult:

    """

    - Suhu awal t0, diturunkan secara geometrik T(k+1) =
alpha * T(k).

    - Pada setiap suhu dibangkitkan iter_per_temp
tetangga acak.

    - Tetangga lebih baik (dE > 0) selalu diterima,
tetangga lebih buruk diterima

      dengan probabilitas exp(dE / T).

    - Berhenti ketika T <= t_min.

    - Solusi akhir adalah state terbaik yang pernah
dikunjungi.
```

```
    """
    if t0 <= 0 or t_min <= 0:
        raise ValueError("t0 dan t_min harus bernilai
positif")
    if not (0 < alpha < 1):
        raise ValueError("alpha harus berada di antara 0
dan 1")


    start_time = time.time()
    current = initial_state.clone()
    current_score = current.get_score()
    initial_score = current_score


    best = current
    best_score = current_score


    score_history: List[float] = [current_score]
    iteration_history: List[int] = [0]
    temp_history: List[float] = []


    prob_history: List[float] = []
    prob_iterations: List[int] = []


    rejected_streak = 0
    stuck_count = 0
    accepted_worse = 0


    T = t0
    iteration = 0
```

```
    while T > t_min:
        for _ in range(iter_per_temp):
            iteration += 1
            neighbor = get_random_neighbor(current)

            accepted = False
            if neighbor is not None:
                neighbor_score = neighbor.get_score()
                delta = neighbor_score - current_score

                if delta > 0:
                    accepted = True
                else:
                    prob = math.exp(delta / T)
                    prob_history.append(prob)
                    prob_iterations.append(iteration)
                    if random.random() < prob:
                        accepted = True
                        if delta < 0:
                            accepted_worse += 1

                if accepted:
                    current = neighbor
                    current_score = neighbor_score

            if accepted:
                rejected_streak = 0
            else:
```

```
                rejected_streak += 1
                if rejected_streak == stuck_limit:
                    stuck_count += 1
                    rejected_streak = 0

            if current_score > best_score:
                best = current
                best_score = current_score

            score_history.append(current_score)
            iteration_history.append(iteration)
            temp_history.append(T)

        T *= alpha

    duration = time.time() - start_time

    return SearchResult(
        algorithm_name="Simulated Annealing",
        initial_state=initial_state,
        final_state=best,
        initial_score=initial_score,
        final_score=best_score,
        iterations=iteration,
        duration_seconds=duration,
        score_history=score_history,
        iteration_history=iteration_history,
        extra_metrics={
```

```
            "t0": t0,
            "alpha": alpha,
            "t_min": t_min,
            "iter_per_temp": iter_per_temp,
            "stuck_limit": stuck_limit,
            "stuck_count": stuck_count,
            "accepted_worse_moves": accepted_worse,
            "last_score": current_score,
            "temperature_history": temp_history,
            "prob_history": prob_history,
            "prob_iterations": prob_iterations,
        },
    )
```

2.5 Genetic Algorithm

Genetic Algorithm (GA) adalah teknik optimasi berbasis populasi yang mereplikasi prinsip seleksi alam, rekombinasi genetik, dan mutasi acak sebagaimana diperkenalkan oleh John Holland [3], [4]. Berbeda dengan metode *single-state search*, GA memelihara sekumpulan kandidat solusi (*population*)

2.5.1 Representasi Kromosom dan Gen

Sesuai prinsip perancangan representasi genetik [3], koordinat ruang kontinu tidak dikodekan langsung melainkan menggunakan representasi berbasis permutasi prioritas pemuatan:

- **Gen (Gene):** Pasangan tuple (i, *orientation_idx*), dengan i menyatakan indeks paket dan $orientation\ idx \in \{0, \ldots, 5\}$ menyatakan orientasi rotasi kartesius [3].
- **Kromosom (*Chromosome*):** Permutasi terurut dari seluruh paket yang tersedia berukuran n:

$$Chromosome = [Gene_0, Gene_1, \ldots, Gene_{n-1}]$$

2.5.2 Heuristik Decoding Bottom-Left-Back (BLB)

Kromosom diterjemahkan menjadi objek `State` fisik melalui fungsi `decode`. Paket dimuat satu per satu ke titik tumpuan kandidat (*candidate anchors)* dengan aturan urutan ruang **Bottom-Left-Back (BLB)** (ketinggian z terendah, kedalaman y terjauh, dan lebar x terkiri) [6]. Fungsi `_can_place` secara ketat memverifikasi bahwa muatan tidak melanggar kapasitas bobot, dinding kontainer, tabrakan balok 3D, tumpuan alas, dan batasan barang pecah-belah. Paket yang tidak muat dibiarkan di luar truk sehingga seluruh konfigurasi hasil *decoding* dijamin 100% valid secara fisika.

2.5.3 Operator Genetika dan Elitisme

Operator genetika yang diimplementasikan mengikuti taksonomi standar GA [3] :

$$P_k = \frac{Fitness_k}{\sum_{j=1}^{pop-size} Fitness_j}$$

1. **Seleksi (*Roulette-Wheel Selection)***: Peluang pemilihan individu sebagai induk proporsional terhadap nilai kebugarannya [3].
2. **Rekombinasi (*Order Crossover - OX)* :** Untuk menjaga integritas permutasi tanpa duplikasi paket, segmen kromosom dari induk pertama di lain ke anak, dan slot yang tersisa
3. **Mutasi (*Mutation)* :** Setiap gen memiliki probabilitas mutation_rate untuk bermutasi, dengan pilihan seimbang menukar urutan paket (*swap*) atau merotasi orientasi [3].
4. **Elitisme (*Elitism)* :** Sebanyak `elite` individu terbaik dipertahankan langsung ke generasi berikutnya tanpa modifikasi, menjamin bahwa kurva *maximum fitness* bersifat monoton tidak menurun (*non-decreasing*) sepanjang generasi [3].
5. **Fitness Caching:** Algoritma memanfaatkan kamus `cache` berbasis hash tuple kromosom untuk mencegah kalkulasi *decoding* geometri ulang pada individu yang identik.

2.5.4 Kode Sumber Implementasi Genetic Algorithm

```
import random

import time

from typing import Callable, Dict, List, Tuple
```

```
from src.core.models import Package, Truck, ORIENTATIONS
from src.core.constraints import (
    is_within_bounds,
    is_overlapping_3d,
    has_horizontal_overlap,
    is_supported,
)
from src.core.objectives import default_objective
from src.core.state import State
from .base import SearchResult


Gene = Tuple[int, int]
Chromosome = List[Gene]



def _can_place(pkg: Package, loaded: List[Package],
truck: Truck, loaded_weight: float) -> bool:
    """Cek kendala lokal paket baru terhadap paket yang
sudah dimuat."""
    if loaded_weight + pkg.weight > truck.max_capacity:
        return False
    if not is_within_bounds(pkg, truck):
        return False
    for other in loaded:
        if is_overlapping_3d(pkg, other):
            return False
    if not is_supported(pkg, loaded):
        return False
```

```
    _, _, h = pkg.current_dims()
    for other in loaded:
        _, _, oh = other.current_dims()
        if other.is_fragile and other.z + oh == pkg.z and
has_horizontal_overlap(pkg, other):
            return False
        if pkg.is_fragile and other.z == pkg.z + h and
has_horizontal_overlap(pkg, other):
            return False
    return True




def _new_anchors(pkg: Package, truck: Truck) ->
List[Tuple[int, int, int]]:
    """Titik kandidat jangkar baru di sisi kanan,
belakang, dan atas paket."""
    w, l, h = pkg.current_dims()
    points = [(pkg.x + w, pkg.y, pkg.z), (pkg.x, pkg.y +
l, pkg.z)]
    if not pkg.is_fragile:
        points.append((pkg.x, pkg.y, pkg.z + h))
    return [(x, y, z) for x, y, z in points if x <
truck.w and y < truck.l and z < truck.h]




def decode(
    chromosome: Chromosome,
    truck: Truck,
    packages: List[Package],
    objective_fn: Callable[[State], float] =
```

```
default_objective,

) -> State:

    """Mengubah kromosom menjadi State yang valid secara
fisika."""

    pkgs = [p.clone() for p in packages]

    for p in pkgs:

        p.is_loaded = False

        p.set_position(0, 0, 0)


    loaded: List[Package] = []

    loaded_weight = 0.0

    anchors = {(0, 0, 0)}


    for idx, ori in chromosome:

        pkg = pkgs[idx]

        pkg.set_orientation(ori)

        for x, y, z in sorted(anchors, key=lambda a:
(a[2], a[1], a[0])):

            pkg.set_position(x, y, z)

            if _can_place(pkg, loaded, truck,
loaded_weight):

                pkg.is_loaded = True

                loaded.append(pkg)

                loaded_weight += pkg.weight

                anchors.discard((x, y, z))

                anchors.update(_new_anchors(pkg, truck))

                break

        if not pkg.is_loaded:

            pkg.set_position(0, 0, 0)
```

```
    return State(truck, pkgs, objective_fn)



def random_chromosome(n: int) -> Chromosome:
    order = list(range(n))
    random.shuffle(order)
    return [(i, random.randrange(len(ORIENTATIONS))) for
i in order]



def roulette_select(population: List[Chromosome],
fitness: List[float]) -> Chromosome:
    """Seleksi roulette-wheel proporsional terhadap
fitness."""
    if sum(fitness) <= 0:
        return random.choice(population)
    return random.choices(population, weights=fitness,
k=1)[0]



def order_crossover(parent1: Chromosome, parent2:
Chromosome) -> Chromosome:
    """Order Crossover (OX) mempertahankan sifat
permutasi paket."""
    n = len(parent1)
    if n < 2:
        return list(parent1)

    a, b = sorted(random.sample(range(n), 2))
```

```
    child: List[Gene] = [None] * n  # type: ignore
    child[a:b + 1] = parent1[a:b + 1]
    taken = {idx for idx, _ in parent1[a:b + 1]}

    rest = [g for g in parent2 if g[0] not in taken]
    empty_slots = [i for i in range(n) if i < a or i > b]
    for i, gene in zip(empty_slots, rest):
        child[i] = gene
    return child



def mutate(chromosome: Chromosome, mutation_rate: float)
-> Chromosome:
    """Mutasi gen: tukar urutan atau rotasi acak."""
    child = list(chromosome)
    n = len(child)
    for i in range(n):
        if random.random() >= mutation_rate:
            continue
        if random.random() < 0.5 and n > 1:
            j = random.randrange(n)
            child[i], child[j] = child[j], child[i]
        else:
            idx, _ = child[i]
            child[i] = (idx,
random.randrange(len(ORIENTATIONS)))
    return child
```

```
def genetic_algorithm(

    truck: Truck,

    packages: List[Package],

    pop_size: int = 30,

    generations: int = 50,

    crossover_rate: float = 0.9,

    mutation_rate: float = 0.1,

    elite: int = 2,

    objective_fn: Callable[[State], float] =
default_objective,

) -> SearchResult:

    if pop_size < 2:

        raise ValueError("pop_size minimal 2")

    elite = max(0, min(elite, pop_size))


    start_time = time.time()

    n = len(packages)

    cache: Dict[Tuple[Gene, ...], float] = {}


    def fitness_of(chrom: Chromosome) -> float:

        key = tuple(chrom)

        if key not in cache:

            cache[key] = float(objective_fn(decode(chrom,
truck, packages, objective_fn)))

        return cache[key]


    population = [random_chromosome(n) for _ in
range(pop_size)]

    fitness = [fitness_of(c) for c in population]
```

```
    best_idx = max(range(pop_size), key=lambda i:
fitness[i])
    initial_chrom = population[best_idx]
    best_chrom = initial_chrom
    best_fit = fitness[best_idx]

    max_history = [max(fitness)]
    avg_history = [sum(fitness) / pop_size]
    iteration_history = [0]

    for gen in range(1, generations + 1):
        ranked = sorted(range(pop_size), key=lambda i:
fitness[i], reverse=True)
        new_pop = [population[i] for i in ranked[:elite]]

        while len(new_pop) < pop_size:
            p1 = roulette_select(population, fitness)
            p2 = roulette_select(population, fitness)
            if random.random() < crossover_rate:
                child = order_crossover(p1, p2)
            else:
                child = list(p1)
            new_pop.append(mutate(child, mutation_rate))

        population = new_pop
        fitness = [fitness_of(c) for c in population]

        gen_best = max(range(pop_size), key=lambda i:
```

```
fitness[i])
        if fitness[gen_best] > best_fit:
            best_fit = fitness[gen_best]
            best_chrom = population[gen_best]

        max_history.append(max(fitness))
        avg_history.append(sum(fitness) / pop_size)
        iteration_history.append(gen)

    duration = time.time() - start_time
    initial_state = decode(initial_chrom, truck,
packages, objective_fn)
    final_state = decode(best_chrom, truck, packages,
objective_fn)

    return SearchResult(
        algorithm_name="Genetic Algorithm",
        initial_state=initial_state,
        final_state=final_state,
        initial_score=initial_state.get_score(),
        final_score=final_state.get_score(),
        iterations=generations,
        duration_seconds=duration,
        score_history=list(max_history),
        iteration_history=iteration_history,
        extra_metrics={
            "pop_size": pop_size,
            "generations": generations,
            "crossover_rate": crossover_rate,
```

```
            "mutation_rate": mutation_rate,
            "elite": elite,
            "max_fitness_history": max_history,
            "avg_fitness_history": avg_history,
            "best_chromosome": [list(g) for g in
best_chrom],
        },
    )
```

2.6 Visualisasi State dan Hasil Eksperimen

2.6.1 Gambaran Umum Visualisasi

Visualisasi diimplementasikan untuk mempermudah pengamatan terhadap konfigurasi penempatan paket dan perkembangan proses pencarian solusi. Sistem visualisasi dibagi menjadi dua komponen utama, yaitu visualisasi tiga dimensi untuk merepresentasikan kondisi fisik peti kemas dan visualisasi grafik untuk menampilkan metrik hasil pencarian algoritma.

Implementasi visualisasi tiga dimensi menggunakan library Plotly, sedangkan grafik metrik menggunakan Matplotlib. Kedua komponen menerima data dari hasil eksekusi algoritma yang direpresentasikan melalui objek `SearchResult`. Objek tersebut menyimpan informasi seperti `initial_state`, `final_state`, `initial_score`, `final_score`, `score_history`, `iteration_history`, `duration_seconds`, serta `extra_metrics` untuk metrik khusus setiap algoritma.

Alur visualisasi dimulai dari pembacaan berkas masukan JSON yang berisi informasi truk dan paket. Program kemudian menjalankan satu algoritma pencarian berdasarkan konfigurasi pengguna melalui fungsi `run_single_search()`. Setelah pencarian selesai, hasilnya disimpan dalam format JSON dan dapat digunakan untuk menghasilkan visualisasi perbandingan state maupun grafik metrik sesuai opsi eksekusi yang dipilih.

2.6.2 Variabel Permasalahan

Visualisasi tiga dimensi diimplementasikan pada berkas `src/visualization/plot3d.py`. Setiap konfigurasi direpresentasikan menggunakan objek State yang berisi dimensi peti kemas, daftar paket yang dimuat, dan daftar paket yang berada di luar peti kemas.

Peti kemas divisualisasikan sebagai kerangka balok (*wireframe*) berwarna hitam yang menunjukkan batas ruang maksimum. Setiap paket direpresentasikan sebagai balok tiga dimensi menggunakan objek `Mesh3d` dari Plotly. Dimensi balok ditentukan melalui method `current_dims()` sehingga bentuk paket mengikuti orientasi yang berlaku pada state tersebut.

Untuk membedakan karakteristik paket, visualisasi menggunakan beberapa warna. Paket biasa yang berada di dalam peti kemas diberi warna berdasarkan palet yang telah ditentukan, sedangkan paket dengan atribut `is_fragile = True` ditampilkan menggunakan warna merah. Paket yang tidak dimuat ditampilkan dalam warna abu-abu dengan transparansi lebih tinggi.

Paket yang berada di luar peti kemas ditempatkan pada area ilustratif di samping kiri truk. Posisi tersebut dihasilkan melalui perhitungan koordinat tampilan `disp_x`, `disp_y`, dan `disp_z`, sehingga tidak merepresentasikan koordinat penempatan sebenarnya pada proses pencarian. Penataan ini bertujuan agar paket yang belum dimuat tetap dapat diamati tanpa mengganggu tampilan konfigurasi di dalam peti kemas.

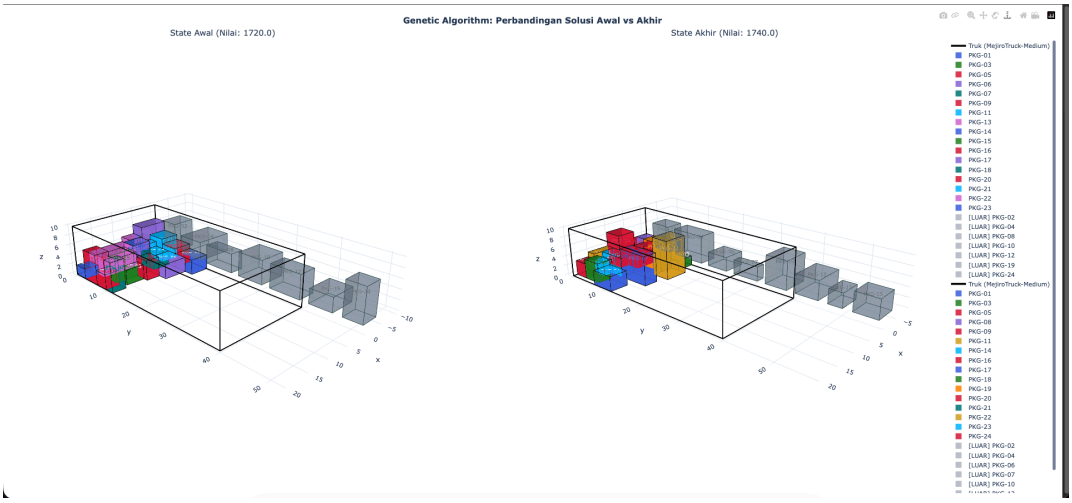
Setiap balok dilengkapi dengan garis tepi dan label `id` paket pada bagian tengahnya. Visualisasi juga mendukung interaksi pengguna melalui fitur *hover* untuk menampilkan informasi tambahan, seperti dimensi setelah orientasi, posisi tampilan, nilai barang, berat, status barang pecah-belah, dan ETA.

2.6.3 Representasi Solusi dan Batasan Permasalahan

Perbandingan konfigurasi awal dan akhir diimplementasikan melalui fungsi `visualize_comparison_3d()`. Fungsi ini menerima dua objek State, yaitu initial_state dan final_state, kemudian menghasilkan dua visualisasi tiga dimensi yang ditempatkan secara berdampingan menggunakan `make_subplots()`.

Bagian kiri menunjukkan konfigurasi sebelum proses pencarian, sedangkan bagian kanan menunjukkan konfigurasi terbaik yang dikembalikan oleh algoritma. Setiap bagian dilengkapi dengan nilai fungsi objektif yang dihitung melalui method `get_score()`.

Perbandingan ini memungkinkan pengguna mengamati perubahan posisi, orientasi, dan status pemuatan paket setelah algoritma dijalankan. Selain melihat perubahan konfigurasi secara visual, pengguna juga dapat membandingkan nilai fungsi objektif pada kedua state untuk mengetahui perbedaan kualitas solusi yang dihasilkan.



Gambar 2.1: Contoh output visualisasi program

Pada gambar, kerangka hitam menunjukkan batas peti kemas, balok berwarna menunjukkan paket yang berhasil dimuat, balok merah menunjukkan paket pecah-belah yang berada di dalam peti kemas, sedangkan balok abu-abu menunjukkan paket yang tidak dimuat. Nilai objektif yang tertera pada masing-masing state menunjukkan kualitas konfigurasi berdasarkan fungsi objektif yang digunakan.

2.6.4 Visualisasi Grafik Hasil Pencarian

Selain visualisasi tiga dimensi, program menyediakan beberapa jenis grafik untuk memperlihatkan perkembangan proses pencarian pada setiap algoritma. Grafik diimplementasikan melalui berkas `src/visualization/metrics.py` menggunakan Matplotlib.

a. Grafik Nilai Fungsi Objektif

Fungsi `plot_objective_history()` digunakan untuk menampilkan perubahan nilai fungsi objektif selama proses pencarian. Data diperoleh dari atribut `score_history`, sedangkan sumbu horizontal menggunakan `iteration_history` apabila tersedia dengan panjang yang sesuai. Jika tidak, indeks riwayat digunakan sebagai pengganti.

Grafik ini digunakan pada algoritma Hill-Climbing dan Simulated Annealing untuk mengamati perkembangan nilai fungsi objektif terhadap banyak iterasi. Pada Simulated Annealing, riwayat tersebut merekam nilai state yang sedang dikunjungi, sehingga grafik dapat mengalami penurunan meskipun solusi akhir yang dikembalikan merupakan state terbaik yang ditemukan selama pencarian.

b. Grafik Probabilitas Penerimaan Simulated Annealing

Fungsi `plot_sa_acceptance()` digunakan untuk menampilkan perkembangan nilai probabilitas penerimaan kandidat solusi yang tidak memberikan peningkatan fungsi objektif pada algoritma Simulated Annealing. Nilai probabilitas diperoleh melalui perhitungan `exp(ΔE/T)`, dengan `ΔE` merupakan selisih nilai objektif kandidat terhadap state saat ini dan T merupakan temperatur.

Data grafik diperoleh dari `prob_history` pada `extra_metrics`, sedangkan posisi iterasi diambil dari `prob_iterations`. Dengan demikian, grafik menunjukkan probabilitas pada iterasi ketika kandidat yang tidak lebih baik dievaluasi, bukan pada seluruh iterasi pencarian.

c. Grafik Fitness Genetic Algorithm

Fungsi `plot_ga_fitness()` digunakan untuk menampilkan perkembangan nilai fitness maksimum dan rata-rata populasi pada setiap generasi Genetic Algorithm.

Grafik menggunakan dua data utama, yaitu `max_fitness` untuk menunjukkan nilai individu terbaik pada setiap generasi dan `avg_fitness` untuk menunjukkan rata-rata nilai fitness seluruh individu dalam populasi.

Kedua kurva ditampilkan pada grafik yang sama untuk mempermudah pengamatan perkembangan kualitas populasi. Perbandingan keduanya membantu memperlihatkan bagaimana individu terbaik dan keseluruhan populasi berkembang selama proses pencarian.

2.6.5 Integrasi dan Penyimpanan Hasil Visualisasi

Integrasi visualisasi dilakukan melalui fungsi `main()` pada berkas `main.py`. Setelah algoritma selesai dijalankan, program menerima objek SearchResult yang selanjutnya digunakan untuk menampilkan ringkasan hasil, menyimpan data pencarian, dan menghasilkan visualisasi apabila opsi terkait diaktifkan.

Hasil eksekusi selalu disimpan dalam format JSON melalui fungsi `save_run_result()` yang terdapat pada berkas `src/utils/io_handler.py`. Data yang disimpan meliputi informasi algoritma, state awal dan akhir, nilai fungsi objektif, riwayat pencarian, durasi komputasi, dan metrik tambahan yang tersedia.

Untuk visualisasi, program menyediakan tiga opsi utama melalui antarmuka *command-line*, yaitu `--plot` untuk membuka visualisasi tiga dimensi, `--save-plot` untuk menyimpan perbandingan state dalam format HTML interaktif, dan `--save-metrics` untuk menyimpan grafik metrik dalam format PNG. Berkas

disimpan pada direktori `output/` secara default, dengan nama yang mengandung identitas algoritma dan waktu eksekusi.

Program menjalankan satu proses pencarian untuk setiap pemanggilan. Oleh karena itu, eksperimen berulang dilakukan melalui beberapa pemanggilan program dengan konfigurasi dan seed yang dicatat secara terpisah.

2.6.6 Integrasi dan Penyimpanan Hasil Visualisasi

Implementasi visualisasi dipisahkan menjadi beberapa fungsi berdasarkan jenis data yang ditampilkan. Pemisahan ini memungkinkan komponen visualisasi digunakan oleh algoritma yang berbeda tanpa perlu mengubah mekanisme pencarian.

Fungsi	Deskripsi
`build_state_traces()`	Mengubah konfigurasi paket dan truk menjadi kumpulan objek visual tiga dimensi.
`visualize_state_3d()`	Membentuk visualisasi tiga dimensi untuk satu objek `State`.
`visualize_comparison_3d()`	Menampilkan perbandingan state awal dan akhir secara berdampingan.
`plot_objective_history()`	Membentuk grafik perkembangan nilai fungsi objektif terhadap iterasi.
`plot_sa_acceptance()`	Membentuk grafik probabilitas penerimaan kandidat pada Simulated Annealing.
`plot_ga_fitness()`	Membentuk grafik fitness maksimum dan rata-rata populasi Genetic Algorithm.

Tabel 2.1 Function utama visualisasi

Fungsi `build_state_traces()` bertugas membangun objek visual berdasarkan posisi dan orientasi masing-masing paket. Fungsi ini digunakan oleh `visualize_state_3d()` dan `visualize_comparison_3d()` sehingga representasi konfigurasi tetap menggunakan mekanisme yang sama.

Untuk grafik metrik, `plot_objective_history()` membaca riwayat skor dari objek `SearchResult`, sedangkan `plot_sa_acceptance()` menggunakan riwayat probabilitas dari `extra_metrics`. Fungsi `plot_ga_fitness()` menerima dua daftar nilai fitness yang dihasilkan selama proses evolusi populasi.

2.7 Fungsi Objektif Alternatif

Dalam teori pemrograman matematis dan optimasi kombinatorial, fungsi objektif (*objective function*) merupakan formula yang mendefinisikan target kuantitatif dari suatu sistem optimasi [5]. Permasalahan permuatan kontainer 3 dimensi di industri logistik modern tidak hanya berfokus pada maksimasi nilai nominal tunggal, melainkan merupakan permasalahan optimasi multi-objektif (*multi-objective optimization*) yang melibatkan pemanfaatan kapasitas ruang dan prioritas batas waktu pengiriman (*delivery deadlines)*, [3].

2.7.1 Formulasi Matematis Fungsi Objektif Alternatif

Fungsi objektif alternatif diimplementasikan pada fungsi `bonus_urgency_volume_objective` dengan mengintegrasikan dua kriteria linear [5] :

$$Skor_{bonus}(s) = \sum_{p \in Loaded} \left(\frac{value_p}{ETA_p + 1.0}\right) + 20.0\, x \left(\frac{\sum_{p \in Loaded} Vol(p)}{Vol(Truck)}\right)$$

Apabila kontainer dalam keadaan kosong atau konfigurasi melanggar batasan fisik, fungsi mengembalikan nilai penalti 0.0.

2.7.2 Rasionalisasi Komponen Objektif

Sesuai dengan konsep pembobotan fungsi objektif dalam optimasi multikriteria, formula ini memadukan dua tujuan strategis tanpa memperkenalkan atribut variabel baru di luar skema berkas JSON :

1. **Bobot Urgensi Kedatangan ($\frac{value}{ETA + 1}$):** Atribut eta merepresentasikan sisa hari toleransi pengantaran ($ETA \geq 1$). Pembagian nilai dengan (ETA + 1) memberikan bobot keutamaan yang lebih besar bagi paket ber-ETA rendah (mendesak), meminimalkan risiko keterlambatan pengiriman [5].

2. **Efisiensi Utilitasi Ruang** (20.0 x $\frac{Vol_{loaded}}{Vol_{truck}}$): Rasio volume paket terhadap volume kontainer mengukur tingkat kerapatan muatan (*packing density*). Koefisien pengali 20.0 ditetapkan untuk menyetarakan skala rasio [0.0, 1.0] terhadap rentang nilai ekonomis barang, mendorong algoritma untuk memadatkan susunan balok dan meminimalkan ruang kosong (*void spaces*) [5].

2.7.3 Integrasi dan Pemanggilan pada CLI

Fungsi objektif alternatif ini dapat dipanggil secara langsung melalui argumen CLI `--objective bonus` pada semua algoritma pencarian lokal :

```
python main.py -i test_cases/small.json -a steepest
--objective bonus
```

Nama Fungsi	Parameter Masukan	Keluaran	Deskripsi Singkat
`default_objective`	`state`	`float`	Menghitung total nilai nominal paket yang berhasil dimuat ke dalam truk.
`bonus_urgency_volume_objective`	`state`	`float`	Menghitung nilai muatan terbobot urgensi ETA ditambah bonus rasio kepadatan volume kontainer.

2.7.4 Kode Sumber Objektif Alternatif

```
from typing import TYPE_CHECKING


if TYPE_CHECKING:
    from .state import State




def default_objective(state: "State") -> float:
    """
    Fungsi objektif default:
    Memaksimalkan total value dari paket-paket yang
berhasil dimuat ke dalam truk.
```

```
    """
    return sum(p.value for p in state.loaded_packages)



def bonus_urgency_volume_objective(state: "State") ->
float:
    """
    Fungsi objektif alternatif (Bonus 1 - 3 Poin):
    Mengoptimalkan nilai muatan berdasarkan:
    1. Urgensi pengiriman (ETA sisa hari lebih kecil ->
bobot nilai lebih tinggi).
    2. Efisiensi utilisasi ruang (rasio volume paket
terhadap volume truk)
       untuk meminimalkan ruang kosong tanpa menambah
variabel baru.
    """
    if not state.loaded_packages:
        return 0.0


    # Komponen 1: Nilai paket terbobot urgensi ETA
    weighted_value = sum(p.value / (p.eta + 1.0) for p in
state.loaded_packages)


    # Komponen 2: Rasio kepadatan ruang (Volume
Utilization)
    truck_vol = state.truck.volume()
    loaded_vol = sum(p.volume() for p in
state.loaded_packages)
    volume_ratio = loaded_vol / truck_vol if truck_vol >
0 else 0.0


    # Bobot pengali volume_ratio disesuaikan agar
```

```
seimbang dengan rentang nilai
    return weighted_value + (20.0 * volume_ratio)
```

Bab III : Hasil dan Analisis Eksperimen

3.1 Konfigurasi dan Skenario Eksperimen

Pengujian menggunakan tiga berkas masukan yang tersedia pada repositori, yaitu test_cases/small.json, medium.json, dan large.json, tanpa mengubah isi ketiganya. Pengujian Hill-Climbing (HC) dan Genetic Algorithm (GA) dilakukan pada ketiga dataset untuk melihat kinerja algoritma pada ukuran permasalahan yang berbeda. Sementara itu, pengujian Simulated Annealing (SA) difokuskan pada medium.json sebagai dataset acuan untuk perbandingan utama antaralgoritma. Seluruh eksperimen menggunakan default_objective, yaitu memaksimalkan jumlah nilai paket yang berhasil dimuat.

Dataset	Paket	Dimensi truk	Kapasitas	Total berat	Total nilai	Fragile
small	6	10 × 20 × 5	100	108	485	2
medium	24	20 × 40 × 10	400	669	2.845	7
large	48	30 × 60 × 15	1.000	1.398	5.970	14

Tabel 3.1 Karakteristik ketiga dataset. Berat dan kapasitas dinyatakan dalam satuan unit berat.

Total berat seluruh paket melampaui kapasitas pada ketiga dataset. Karena itu, pencarian perlu memilih subset paket dan menyusun penempatannya agar seluruh kendala tetap terpenuhi. Perbedaan dataset mencakup jumlah paket, dimensi truk, kapasitas, dan karakteristik muatan; perbandingan ini tidak mengisolasi pengaruh jumlah paket saja.

Dataset medium.json dipilih sebagai kasus uji utama Simulated Annealing karena memiliki tingkat kompleksitas yang cukup untuk mengamati proses pencarian solusi. Dataset small.json memiliki jumlah paket yang relatif sedikit sehingga ruang pencariannya lebih sederhana, sedangkan large.json memiliki kompleksitas lebih tinggi dan berpotensi membutuhkan waktu komputasi lebih besar. Dengan 24 paket, termasuk 7 paket fragile, serta kapasitas truk yang terbatas, dataset medium memberikan kondisi yang sesuai untuk mengamati pengaruh mekanisme penerimaan solusi dan penurunan temperatur pada SA.

Selain itu, penggunaan dataset yang sama dengan pengujian utama HC dan GA bertujuan menjaga kesetaraan kondisi perbandingan (fair comparison). Oleh karena itu, SA dijalankan sebanyak tiga kali menggunakan seed yang berbeda pada medium.json. Pengujian SA pada small.json dan large.json tidak termasuk dalam cakupan eksperimen ini, sehingga analisis SA dibatasi pada dataset medium dan tidak digunakan untuk menyimpulkan skalabilitas SA pada ukuran dataset lainnya.

Aspek	Konfigurasi eksperimen
Repetisi	Seed 1, 2, dan 3 untuk setiap konfigurasi pada setiap dataset.
Steepest dan Sideways	max_iterations=300; sample_neighbors=25; Sideways max_sideways=50.
Stochastic	max_iterations=300; satu kandidat per iterasi dengan max_tries=10.
Random Restart	max_restarts=5 inner run; iterations_per_restart=100; sample_neighbors internal=15.
Simulated Annealing	t0=200; alpha=0,95; t_min=0,01; iter_per_temp=10; stuck_limit=50.
GA studi generasi	pop_size=30; generations=20, 50, 100; masing-masing tiga repetisi.
GA studi populasi	generations=50; pop_size=10, 30, 60; masing-masing tiga repetisi.
Parameter GA lain	crossover_rate=0,9; mutation_rate=0,1 per gen; elite=2.
Lingkungan	Windows 11; Intel Core i5-11400H 2,70 GHz; Python 3.12. Pencarian berurutan pada mesin yang sama.
Visualisasi	Matplotlib 3.11.2; Plotly 7.1.0; PNG 3D dari tampilan HTML program melalui Chrome headless.

Tabel 3.2 Konfigurasi parameter eksperimen HC dan GA pada ketiga dataset serta SA pada dataset medium.

Pengujian terdiri atas 12 run HC dan 18 run GA pada masing-masing dataset small, medium, dan large. Khusus Simulated Annealing, dilakukan tiga run pada dataset medium menggunakan seed 1, 2, dan 3. Dengan demikian, jumlah keseluruhan eksperimen adalah 93 run, terdiri atas 36 run HC, 3 run SA, dan 54 run GA. Dua studi GA masing-masing memiliki tiga konfigurasi dengan tiga repetisi. Konfigurasi P30 G50 dijalankan secara terpisah pada kedua studi dan tidak digabung menjadi enam repetisi. Prefiks S, M, dan L pada ID run menunjukkan small, medium, dan large.

Sebanyak 33 hasil pengujian pada dataset medium digunakan kembali dari eksperimen yang telah diverifikasi sebelumnya, terdiri atas 12 run HC, 3 run SA, dan 18 run GA. Selanjutnya, sebanyak 60 run tambahan dilakukan pada dataset small dan large, masing-masing terdiri atas 12 run HC dan 18 run GA. Seluruh pengujian menggunakan source code, parameter algoritma, dan versi Python yang sama. Kedua sesi dijalankan pada mesin yang sama, meskipun durasi eksekusi tetap

dapat dipengaruhi oleh perbedaan beban sistem saat pengambilan data. Kedua sesi memakai mesin yang sama; durasi tetap dipengaruhi beban sistem saat pengambilan data.

Pencarian dipanggil melalui run_single_search() dalam proses Python terpisah per run. Visualisasi dibuat setelah pencarian selesai. Durasi berasal dari duration_seconds_unrounded; HC dan SA tidak memasukkan pembangkitan state awal, Random Restart memasukkan inisialisasi tiap inner run, sedangkan GA memasukkan pembentukan populasi awal tetapi berhenti menghitung waktu sebelum decode keluaran.

Bukti seluruh 99 run tersimpan pada output/laporan_tiga_testcase/<dataset>/<ID>/ berupa result.json, states.html, states.png, dan convergence.png; SA menambahkan acceptance.png.. Gambar pada laporan hanya menampilkan contoh dari medium.json. Hasil visualisasi lainnya, termasuk seluruh run small.json dan large.json, dapat dilihat pada folder output/laporan_tiga_testcase/small/ dan output/laporan_tiga_testcase/large/.

3.2 Hasil dan Analisis Eksperimen Hill-Climbing

Dataset	Varian	Rerata akhir	Min–maks.	Simp. baku	Rerata detik	Rerata iterasi*
small	Steepest	445,00	445–445	0,00	0,0149	2,33
small	Sideways	445,00	445–445	0,00	0,5784	52,33
small	Stochastic	445,00	445–445	0,00	0,0717	300,00
small	Random Restart	445,00	445–445	0,00	0,0380	15,67
medium	Steepest	1.691,67	1.670–1.710	16,50	0,1957	1,67
medium	Sideways	1.720,00	1.715–1.730	7,07	9,9763	75,33
medium	Stochastic	1.705,00	1.690–1.725	14,72	1,3645	300,00
medium	Random Restart	1.715,00	1.710–1.725	7,07	1,0326	13,33
large	Steepest	4.260,00	4.255–4.265	4,08	2,0064	1,67
large	Sideways	4.278,33	4.260–4.300	16,50	92,7451	103,67
large	Stochastic	4.271,67	4.260–4.280	8,50	11,6195	300,00

large	Random Restart	4.266,67	4.260–4.275	6,24	6,8660	12,00

Tabel 3.3 Ringkasan tiga repetisi HC per varian dan dataset. Simpangan baku memakai pembagi n=3.

Pada small, keempat varian mencapai rata-rata skor akhir yang sama, yaitu 445,00, dengan simpangan baku nol. Steepest memerlukan rata-rata 2,33 iterasi, sedangkan Stochastic tetap menjalankan 300 iterasi. Kesamaan skor memperlihatkan bahwa tambahan iterasi tidak selalu memberi perbaikan pada kasus ini.

Pada medium, rata-rata HC tertinggi diperoleh Sideways sebesar 1.720,00. Varian tercepat adalah Steepest dengan durasi rata-rata 0,1957 detik. Hasil ini terbatas pada tiga seed yang diuji.

Pada large, rata-rata HC tertinggi diperoleh Sideways sebesar 4.278,33. Varian tercepat adalah Steepest dengan durasi rata-rata 2,0064 detik. Hasil ini terbatas pada tiga seed yang diuji.

3.2.1 Metrik Khusus dan Interpretasi Penghitung

Run	Iterasi	Batas	Datar tercatat	Datar diterima	Rentetan akhir
S-HC-SW-01	52	50	51	50	51
S-HC-SW-02	53	50	51	50	51
S-HC-SW-03	52	50	51	50	51
M-HC-SW-01	85	50	80	79	51
M-HC-SW-02	81	50	75	74	51
M-HC-SW-03	60	50	58	57	51
L-HC-SW-01	165	50	157	156	51
L-HC-SW-02	88	50	85	84	51
L-HC-SW-03	58	50	57	56	51

Tabel 3.4 Metrik Sideways pada sembilan run. Batas 50 berlaku pada satu rentetan perpindahan datar.

Pada penghentian karena batas sideways, percobaan datar ke-51 sudah masuk penghitung tetapi transisinya tidak diterima. Karena itu, nilai total_sideways_performed perlu dibedakan dari jumlah transisi datar yang diterima. Peningkatan skor mengatur ulang consecutive_sideways sehingga total sideways selama pencarian dapat melampaui 50.

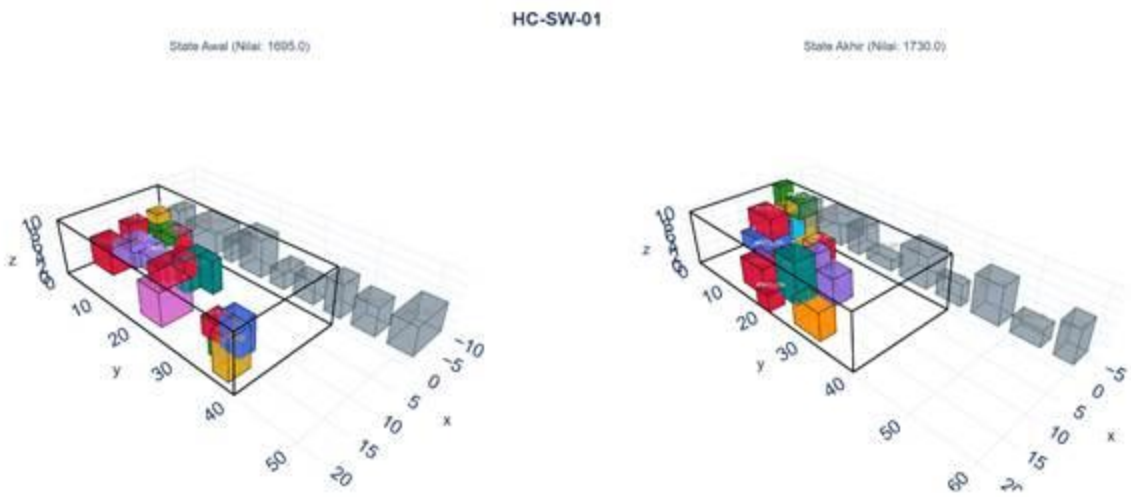
Run	Inner run	Iterasi tiap inner run	Jumlah iterasi dalam	Entri riwayat
S-HC-RR-01	5/5	2, 2, 1, 3, 1	9	14
S-HC-RR-02	5/5	3, 2, 2, 1, 3	11	16
S-HC-RR-03	5/5	2, 2, 2, 1, 5	12	17
M-HC-RR-01	5/5	1, 3, 1, 1, 2	8	13
M-HC-RR-02	5/5	3, 1, 1, 2, 1	8	13
M-HC-RR-03	5/5	2, 2, 1, 3, 1	9	14
L-HC-RR-01	5/5	2, 2, 1, 1, 1	7	12
L-HC-RR-02	5/5	2, 1, 3, 1, 2	9	14
L-HC-RR-03	5/5	1, 1, 1, 1, 1	5	10

Tabel 3.5 Iterasi Random Restart pada sembilan run. Lima titik awal ikut dihitung dalam riwayat gabungan.

Random Restart menjalankan lima pencarian dalam dan menyimpan hasil terbaik di antaranya. Grafik gabungannya dapat turun pada pergantian state awal. Steepest dan Sideways memilih yang terbaik dari sampel tetangga, sehingga terminasi karena tidak ada peningkatan pada sampel belum membuktikan optimum lokal untuk seluruh ruang penempatan. Stochastic menerima peningkatan ketat dari satu kandidat per iterasi dan tetap berjalan hingga batas 300.

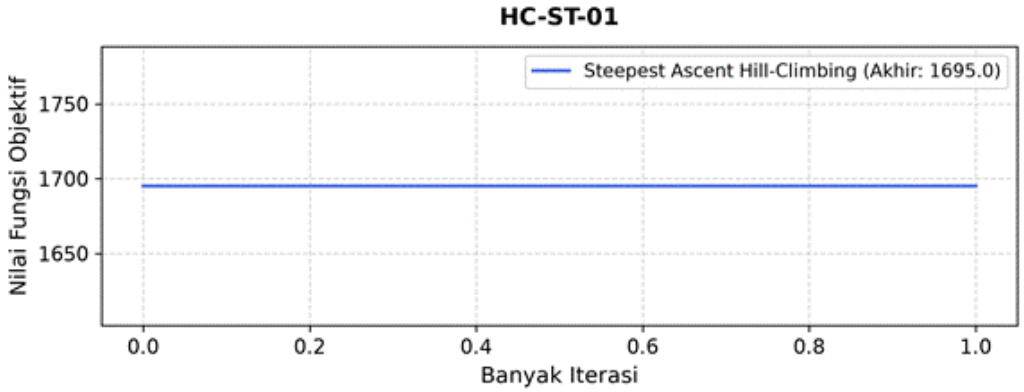
3.2.2 Perilaku Pencarian

Sideways dapat mengubah konfigurasi tanpa mengurangi nilai muatan, sehingga membuka peluang perbaikan berikutnya dengan biaya evaluasi tambahan. Random Restart mencoba beberapa titik awal, sedangkan Stochastic membatasi kandidat yang dinilai pada setiap iterasi. Perbedaan hasil perlu dibaca bersama anggaran pencarian masing-masing, karena jumlah evaluasi tetangga tidak disamakan.



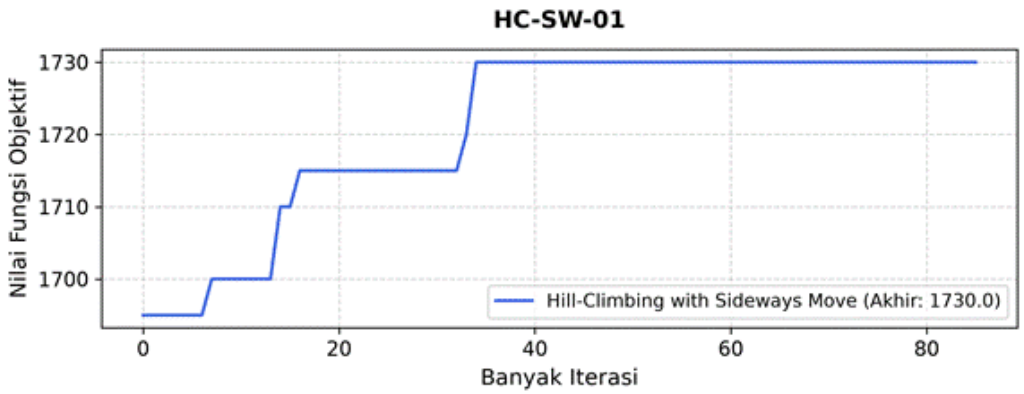
Gambar 3.1 Contoh state awal dan akhir M-HC-SW-01 pada medium, seed 1; skor 1.695 → 1.730.

Gambar 3.2–3.5 memperlihatkan perkembangan nilai objektif keempat varian HC pada medium.json dengan seed 1. Seluruh grafik berasal dari fungsi plot_objective_history() pada program dan menggunakan riwayat pencarian yang sama dengan hasil eksperimen pada tabel sebelumnya.



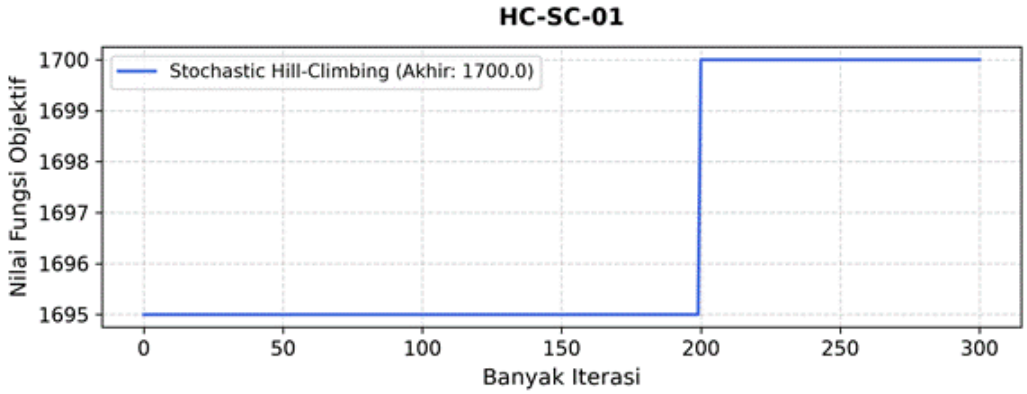
Gambar 3.2 Grafik nilai objektif Steepest Ascent, M-HC-ST-01, pada medium.json dengan seed 1.

Pada M-HC-ST-01, skor tetap 1.695 hingga pencarian berhenti pada iterasi pertama karena kandidat terbaik dari sampel tidak memberikan peningkatan. Garis datar pada grafik menunjukkan tidak adanya perubahan skor pada run ini.



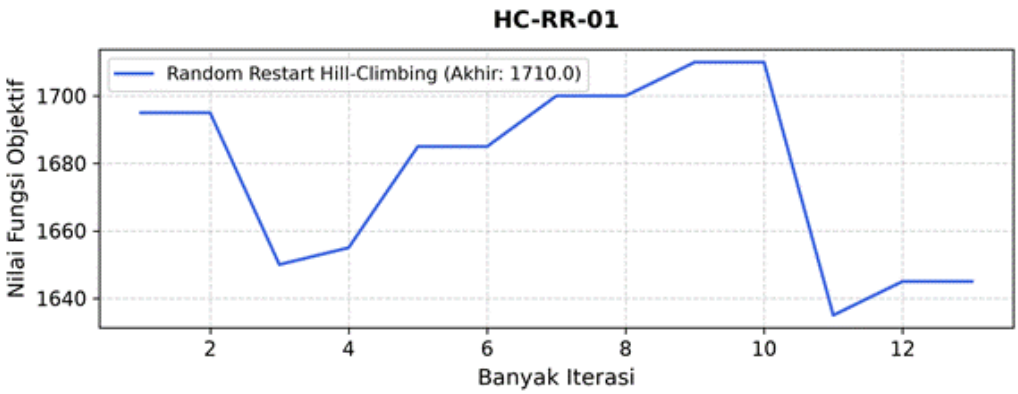
Gambar 3.3 Grafik nilai objektif Sideways Move, M-HC-SW-01, pada medium.json dengan seed 1.

Pada M-HC-SW-01, skor meningkat dari 1.695 menjadi 1.730 pada iterasi ke-34, kemudian mendatar hingga pencarian berhenti pada iterasi ke-85. Bagian mendatar menunjukkan skor yang tetap, meskipun konfigurasi masih dapat berubah melalui perpindahan sideways.



Gambar 3.4 Grafik nilai objektif Stochastic Hill-Climbing, M-HC-SC-01, pada medium.json dengan seed 1.

Pada M-HC-SC-01, hanya terjadi satu perbaikan, yaitu dari 1.695 menjadi 1.700 pada iterasi ke-200. Skor kemudian tetap hingga iterasi ke-300 karena kandidat berikutnya tidak memberikan peningkatan yang diterima.



Gambar 3.5 Grafik nilai objektif Random Restart, M-HC-RR-01, pada medium.json dengan seed 1. Sumbu horizontal dibaca sebagai urutan rekaman gabungan.

Kurva M-HC-RR-01 menggabungkan lima inner run, sehingga skor dapat turun ketika pencarian dimulai kembali dari state awal yang baru. Sumbu horizontal menunjukkan urutan rekaman gabungan, termasuk lima titik awal, bukan penjumlahan iterasi pencarian dalam. Skor tertinggi 1.710 ditemukan pada inner run keempat, sedangkan rekaman terakhir bernilai 1.645. Solusi akhir tetap memakai skor terbaik 1.710.

Skor, durasi, iterasi, state awal dan akhir, serta kurva setiap run HC pada ketiga dataset tersedia pada output/laporan_tiga_testcase/<dataset>/<ID>/.

3.3 Hasil dan Analisis Eksperimen Simulated Annealing

Kasus Simulated Annealing (SA) dievaluasi melalui 3 kali pengulangan independen dengan variasi benih acak (random seed) 1, 2, 3 pada **skenario kasus uji menengah.**

3.3.1 Karakteristik Kasus Uji yang digunakan

Kasus uji yang dijadikan acuan pengujian memiliki spesifikasi fisik sebagai berikut :

- **Peti Kemas / Truk:** Berdimensi lebar W=20,, panjang L=40, dan tinggi H=10 satuan panjang (volume total kontainer sebesar 8.000 satuan volume), dengan kapasitas daya angkut bobot maksimum sebesar 400,0 satuan bobot.
- **Kumpulan Paket:** Terdiri atas 24 paket barang dengan variasi dimensi ukuran, nilai prioritas pengiriman (ETA), dan nilai ekonomis. Total berat kumulatif dari seluruh 24 paket mencapai **669,0 satuan bobot** dengan total nilai keseluruhan sebesar **2.845,0**.
- **Tantangan Optimasi:** Karena total berat seluruh paket melampaui kapasitas angkut truk (669,0>400,0), algoritma tidak dapat memuat seluruh barang sekaligus. Algoritma dituntut secara cerdas memilih kombinasi subset paket yang menghasilkan nilai keuntungan terbesar serta memadatkan tata letak geometrinya.

- **Kendala Khusus:** Sebanyak 7 dari 24 paket berstatus barang pecah-belah (*fragile*) yang tidak boleh ditumpuk atau dijadikan tumpuan bagi paket lain di atasnya.

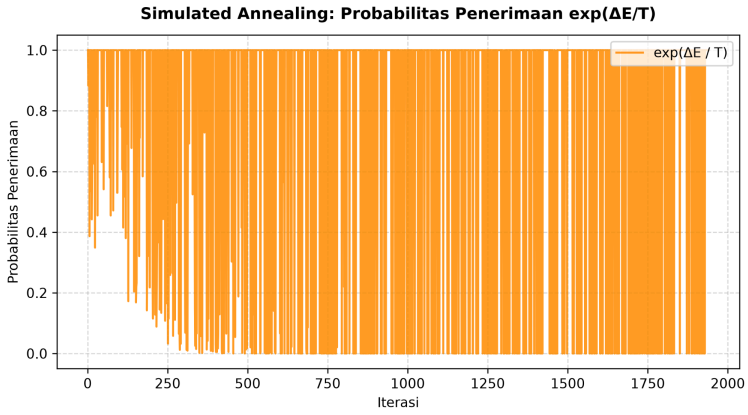
Konfigurasi parameter Simulated Annealing yang diterapkan meliputi: temperatur awal $T_0 = 200,0$, koefisien pendinginan $\alpha = 0.95$, batas suhu akhir $T_{min} = 0,01$, jumlah iterasi per tingkat suhu `iter_per_temp` = 10, dan ambang toleransi penolakan berturut-turut `stuck_limit` = 50. Dengan skema pendinginan geometrik, pencarian menempuh 194 tingkat penurunan suhu atau tepat 1.940 iterasi evaluasi per *run*.

3.3.2 Tabel Rekapitulasi Hasil Eksperimen SA

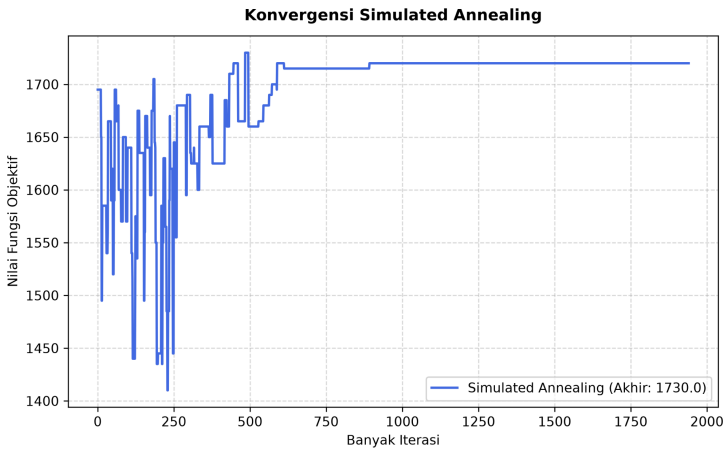
ID Run	Seed	Skor Awal	Skor Akhir	Total Iterasi	Durasi (Detik)	Paket Dimuat	Beban Truk/Kapasitas	Solusi Buruk Diterima	Frekuensi Stuck	Status Fisik
SA-01	1	1695.00	1730.00	1940	3.8546	16 / 24	400.0 / 400.0	43	0	Valid
SA-02	2	1655.00	1720.00	1940	2.4027	15 / 24	400.0 / 400.0	42	0	Valid
SA-03	3	1655.00	1715.00	1940	3.2723	15 / 24	398.0 / 400.0	37	0	Valid
Rata-rata	-	**1668.33**	**1721.67**	**1940**	**3.1765**	**15.33 / 24**	**399.33 / 400.0**	**40.67**	**0**	**100% Valid**

Tabel 3.3 Rekapitulasi Hasil Eksperimen Simulated Annealing pada Kasus Uji

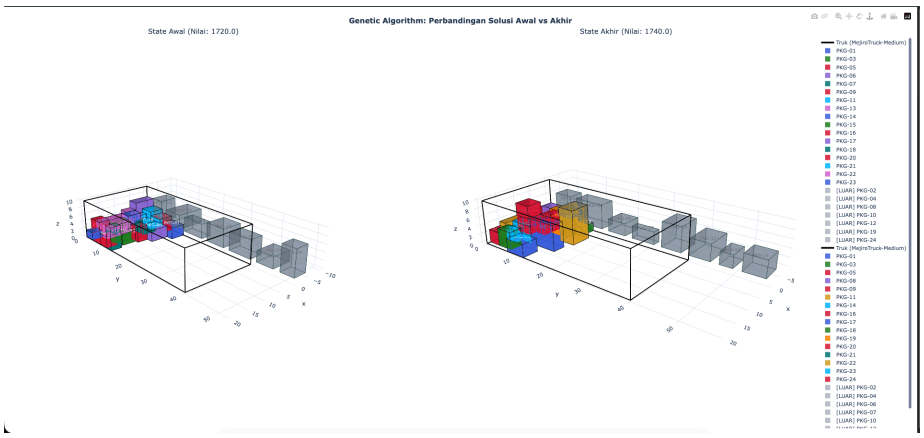
3.3.3 Bukti Visualisasi State dan Grafik Metrik SA



Gambar 3.1: Kurva Fluktuasi Probabilitas Penerimaan Solusi Buruk $exp(\Delta E/T)$ terhadap iterasi pada Simulated Annealing (Run SA-01)



Gambar 3.2: Kurva Konvergensi Nilai Fungsi Objektif terhadap Banyak Iterasi pada Simulated Annealing (Run SA-01).



Gambar 3.3: Visualisasi 3D Pemuatan Kontainer Solusi Awal vs Solusi Akhir pada Simulated Annealing (Run SA-01).

Keterangan: Kiri merupakan State Awal acak (Skor: 1.695,0, 15 paket termuat). Kanan merupakan State Akhir optimal (Skor: 1.730,0, 16 paket termuat, beban 400,0 / 400,0 kapasitas). Balok merah menandakan barang pecah-belah (is_fragile), dan balok transparan abu-abu di luar kontainer menandakan paket yang tidak termuat.

3.3.4 Analisis Karakteristik dan Perilaku Pencarian SA

Berdasarkan data kuantitatif pada Tabel 3.3 dan visualisasi pada Gambar 3.1–3.3, diperoleh analisis komprehensif sebagai berikut:

1. **Efektivitas Optimasi dan Pemenuhan Kendala Fisik**: Algoritma Simulated Annealing berhasil mengoptimalkan susunan muatan dari rata-rata awal 1.668,33 menjadi 1.721,67 (+53,34 poin). Seperti terlihat pada Gambar 3.1, pada solusi akhir (kanan), paket tersusun rapat memenuhi ruang peti kemas hingga mencapai

kapasitas angkut maksimum (400,0 dari 400,0 kg pada SA-01 dan SA-02). Yang terpenting, seluruh barang pecah-belah (balok merah) berhasil diposisikan pada lapisan paling atas atau berdiri sendiri tanpa ditumpuk oleh paket lain, mengonfirmasi bahwa penanganan kendala fisik berjalan 100% valid.

2. **Dinamika Kriteria Metropolis dan Probabilitas Penerimaan**: Gambar 3.3 memperlihatkan secara jelas perilaku probabilitas penerimaan $exp(\Delta E\ /T)$. Pada 500 iterasi pertama (fase temperatur tinggi), nilai probabilitas berfluktuasi tinggi pada rentang 0,40 hingga 0,90. Hal ini tercermin pada Gambar 3.2, di mana kurva nilai objektif mengalami dinamika naik-turun yang agresif karena algoritma secara aktif menerima solusi yang lebih buruk (rata-rata 40,67 kali per run) guna meloloskan diri dari jebakan optimum lokal (*local optima*).

3. **Pembekuan Sistem dan Konvergensi Stabil**: Memasuki iterasi ke-1.000 ke atas saat suhu mendingin menuju $T_{min} = 0,01$, probabilitas penerimaan solusi buruk meluruh drastis mendekati nol (Gambar 3.3). Sistem bertransisi menjadi eksploitatif murni, di mana hanya solusi membaik yang diterima. Hal ini menyebabkan kurva nilai objektif pada Gambar 3.2 mengunci solusi terbaiknya secara stabil di skor 1.730,00 hingga iterasi ke-1.940.

4. **Reliabilitas dan Ketiadaan Hambatan**: Nilai `stuck_count` bernilai 0 di seluruh pengulangan, membuktikan bahwa algoritma tidak pernah mengalami macet (*stuck*) sebanyak 50 kali berturut-turut. Durasi komputasi rata-rata 3,17 detik menunjukkan algoritma sangat responsif untuk ruang pencarian dimensi menengah.

3.4 Hasil dan Analisis Eksperimen Genetic Algorithm

Pengujian Genetic Algorithm (GA) dianalisis melalui dua studi parameter independen.

1. **Studi Parameter 1:** Pengaruh jumlah generasi (`generations` = 20, 50, dan 100) dengan ukuran populasi tetap (`pop_size` = 30)

2. **Studi Parameter 2:** Pengaruh ukuran populasi (`pop_size` = 10, 30, dan 60) dengan jumlah generasi tetap (`generations` = 50)

Parameter konstan P_c = 0.9 (*Order Crossover*), P_m = 0.1, `elite` = 2, fungsi objektif default, dan 3 kali repetisi independen per konfigurasi.

3.4.1 Studi Parameter 1: Variasi Jumlah Generasi (Populasi Tetap = 30)

Konfigurasi Generasi	Seed	Skor Awal	Skor Akhir	Durasi (detik)	Paket Dimuat	Beban Truk	Fitness Maksimum Akhir	Rata-rata Fitness Populasi
Generasi	1	1715.00	1730.00	0.1154	15 / 24	400.0	1730.00	1690.17

20								
	2	1725.00	1735.00	0.1023	14 / 24	400.0	1735.00	1696.83
	3	1715.00	1730.00	0.1102	15 / 24	400.0	1730.00	1692.83
Rata-rata Gen 20	-	*1718.33*	*1731.67*	*0.1093*	*14.67 / 24*	*400.0*	*1731.67*	*1693.28*
Generasi 50	1	1715.00	1740.00	0.2745	15 / 24	400.0	1740.00	1698.83
	2	1725.00	1735.00	0.2453	14 / 24	400.0	1735.00	1695.67
	3	1715.00	1735.00	0.2575	16 / 24	400.0	1735.00	1688.83
Rata-rata Gen 50	-	*1718.33*	*1736.67*	*0.2591*	*15.00 / 24*	*400.0*	*1736.67*	*1694.44*
Generasi 100	1	1715.00	1740.00	0.5318	15 / 24	400.0	1740.00	1698.17
	2	1725.00	1740.00	0.5260	16 / 24	400.0	1740.00	1699.33
	3	1715.00	1735.00	0.5073	16 / 24	400.0	1735.00	1687.33
Rata-rata Gen 100	-	*1718.33*	*1738.33*	*0.5217*	*15.67 / 24*	*400.0*	*1738.33*	*1694.94*

Tabel 3.4: Hasil Eksperimen GA Studi 1 (Variasi Generasi, Populasi = 30)

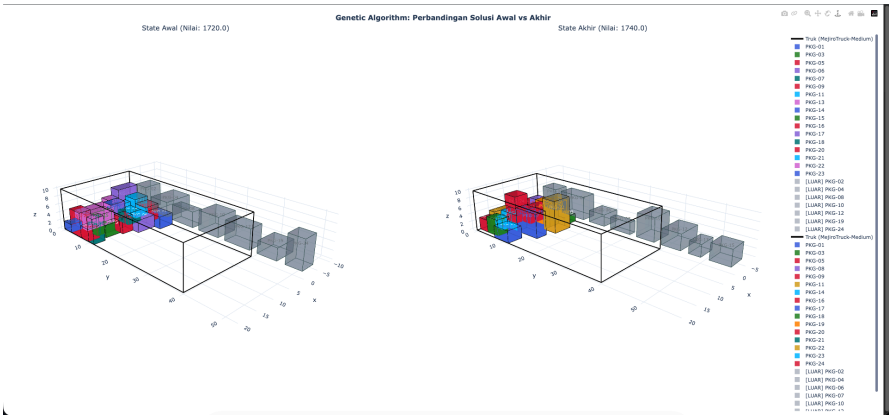
3.4.2 Studi Parameter 2: Variasi Ukuran Populasi (Generasi Tetap = 50)

Konfigurasi Generasi	Seed	Skor Awal	Skor Akhir	Durasi (detik)	Paket Dimuat	Beban Truk	Fitness Maksimum Akhir	Rata-rata Fitness Populasi
Generasi 20	1	1715.00	1730.00	0.1154	15 / 24	400.0	1730.00	1690.17
	2	1725.00	1735.00	0.1023	14 / 24	400.0	1735.00	1696.83
	3	1715.00	1730.00	0.1102	15 / 24	400.0	1730.00	1692.83
Rata-rata Gen 20	-	*1718.33*	*1731.67*	*0.1093*	*14.67 / 24*	*400.0*	*1731.67*	*1693.28*
Generasi 50	1	1715.00	1740.00	0.2745	15 / 24	400.0	1740.00	1698.83
	2	1725.00	1735.00	0.2453	14 / 24	400.0	1735.00	1695.67
	3	1715.00	1735.00	0.2575	16 / 24	400.0	1735.00	1688.83
Rata-rata Gen 50	-	*1718.33*	*1736.67*	*0.2591*	*15.00 / 24*	*400.0*	*1736.67*	*1694.44*
Generasi	1	1715.00	1740.00	0.5318	15 / 24	400.0	1740.00	1698.17

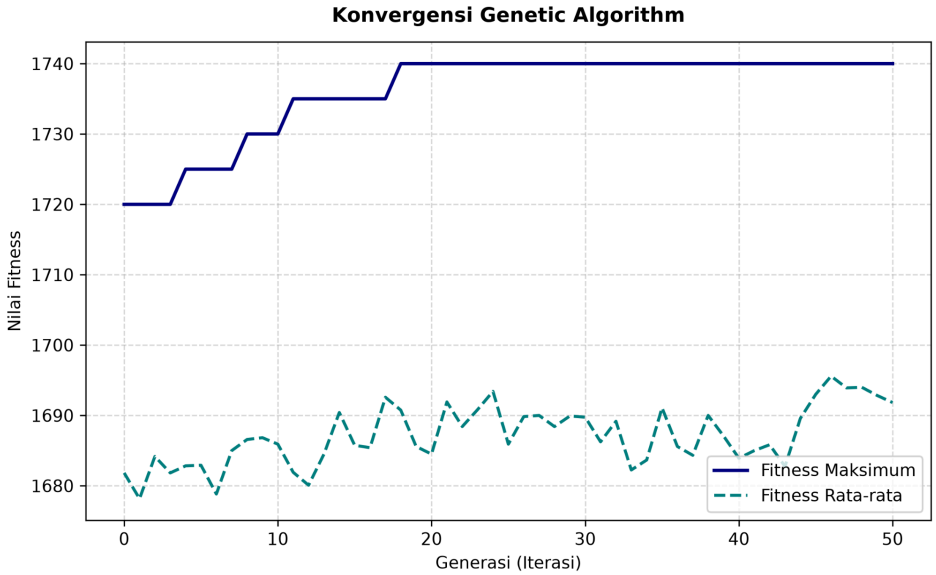
100	2	1725.00	1740.00	0.5260	16 / 24	400.0	1740.00	1699.33
	3	1715.00	1735.00	0.5073	16 / 24	400.0	1735.00	1687.33
Rata-rata Gen 100	-	*1718.33*	*1738.33*	*0.5217*	*15.67 / 24*	*400.0*	*1738.33*	*1694.94*

Tabel 3.5: Hasil Eksperimen GA Studi 2 (Variasi Populasi, Generasi = 50)

3.4.3 Bukti Visualisasi State dan Grafik Metrik GA



Gambar 3.4: Visualisasi 3D Konfigurasi Muatan Solusi Awal vs Solusi Akhir Terbaik Genetic Algorithm (Populasi 60, Generasi 50, Skor Akhir: 1.740,00).



Gambar 3.5: Kurva Fitness Maksimum (Garis Biru Navy) vs Rata-rata Populasi (Garis Putus-Putus Teal) per Generasi pada Genetic Algorithm (Populasi 60, Generasi 50, Seed 1).

3.4.4 Analisis Hasil Eksperimen dan Visualisasi GA

1. **Kualitas Solusi Tertinggi Global**: Genetic Algorithm dengan populasi 60 menghasilkan solusi terbaik mutlak di antara seluruh algoritma yang diuji, mencapai skor 1.740,00 secara konsisten di seluruh repetisi (deviasi standar $\sigma = 0,00$). Seperti terlihat pada Gambar 3.4, kombinasi orientasi dan urutan pemuatan yang dihasilkan oleh operator genetika berhasil menyusun balok secara presisi dari lantai dasar z = 0 menyandar ke dinding belakang tanpa celah horizontal yang terbuang.

2. **Karakteristik Kurva Evolusi Populasi (Gambar 3.5):** Pada Gambar 3.5, kurva fitness maksimum (garis biru tua) menunjukkan sifat monoton tidak pernah menurun (*non-decreasing*) sebagai konsekuensi langsung dari penerapan mekanisme elitisme (elite = 2). Peningkatan fitness berlangsung pesat pada 10 generasi awal (dari 1.720,00 menjadi 1.740,00), lalu stabil hingga generasi ke-50. Di sisi lain, kurva rata-rata populasi (garis putus-putus teal) berfluktuasi stabil pada rentang 1.670–1.700, mengonfirmasi bahwa populasi tetap memiliki diversitas genetik berkat operator mutasi (P_m = 0.1) dan tidak mengalami kejenuhan mutlak (*premature stagnation*).

3. **Perbandingan Efek Parameter: Ukuran Populasi Vs Jumlah Generasi :**

    - **Ukuran Populasi Memberikan Dampak Kualitas Lebih Signifikan:** Pada populasi kecil (pop_size = 10), algoritma cepat terjebak pada optimum lokal dengan rata-rata skor akhir 1.730,00 karena keterbatasan keragaman genetik. Peningkatan populasi menjadi 60 secara langsung mendongkrak skor akhir mencapai nilai optimal global 1.740,00.

    - **Penambahan Generasi Mengalami *Diminishing Returns*:** Pada populasi tetap 30, peningkatan generasi dari 20 ke 50 menaikkan rata-rata skor sebesar +5,00 poin, namun penambahan lebih lanjut dari 50 ke 100 generasi hanya menghasilkan kenaikan tipis +1,66 poin karena ruang pencarian di sekitar individu terbaik telah tereksploitasi penuh.

4. **Efisiensi Waktu Eksekusi:** Berkat arsitektur *decoder* yang efisien dan pemanfaatan *fitness caching* pada kromosom duplikat/elit, Genetic Algorithm menyelesaikan 50 generasi pada populasi 60 hanya dalam waktu **~0,52 detik**. Durasi ini sekitar 6 kali lebih cepat dibandingkan Simulated Annealing (~3,17 detik), sekaligus menghasilkan nilai objektif akhir yang lebih superior (1.740,00 vs 1.721,67).

3.5 Analisis dan Perbandingan Antaralgoritma

3.5.1 Kualitas Solusi terhadap Batas Atas

Dataset	Rerata HC tertinggi	Rerata SA	GA P30 G50	GA P60 G50
small	445,00	445,00	445,00	445,00
medium	1.720,00	1.721,67	1.736,67	1.740,00
large	4.278,33	4.291,67	4.315,00	4.311,67

Tabel 3.9 Perbandingan rata-rata skor akhir. GA P30 G50 memakai studi generasi sebagai konfigurasi acuan.

Skor valid terbaik yang ditemukan pada small, medium, dan large berturut-turut adalah 445, 1.740, 4.320. Nilai ini merupakan hasil terbaik pada matriks pengujian, bukan bukti optimum global. Tabel 3.3, 3.7, dan 3.8 menunjukkan variasi hasil antar-seed, sedangkan Tabel 3.6 memuat hasil individual SA.

Skor absolut small, medium, dan large tidak dibandingkan sebagai ukuran keunggulan algoritma karena total nilai paket dan kapasitasnya berbeda. Perbandingan kualitas dilakukan di dalam dataset yang sama. Hasil yang seragam pada small menunjukkan bahwa dataset tersebut belum banyak membedakan konfigurasi yang diuji; medium dan large memberi pengamatan tambahan terhadap variasi hasil dan biaya pencarian.

SA memperoleh rata-rata sedikit di atas varian HC terbaik pada medium dan large. GA P30 G50 dan P60 G50 menghasilkan rata-rata yang lebih tinggi pada kedua dataset tersebut, tetapi konfigurasi GA terbaik di antara keduanya berbeda menurut dataset. Pada medium, P60 G50 mencapai 1.740 pada seluruh seed; pada large, P30 G50 mempunyai rata-rata lebih tinggi, sedangkan P60 G50 mempunyai simpangan baku lebih kecil. Kualitas dan konsistensi perlu dinilai bersama.

3.5.2 Waktu, Konsistensi, dan Perilaku Algoritma

Durasi rata-rata terendah di antara konfigurasi yang diuji diperoleh Steepest pada small (0,0149 detik); GA P10 G50 pada medium (0,1662 detik); GA P10 G50 pada large (0,9631 detik).

Durasi rata-rata Sideways meningkat dari 0,5784 detik pada small menjadi 9,9763 detik pada medium dan 92,7451 detik pada large. Untuk SA, nilainya berturut-turut 0,4332, 8,8720, dan 57,1423 detik. Kedua metode memerlukan waktu lebih banyak daripada Steepest dan konfigurasi GA yang diuji pada setiap dataset, dengan hasil skor yang perlu dibaca bersama tabel sebelumnya.

Penambahan paket menambah pekerjaan validasi dan penempatan, tetapi durasi juga dipengaruhi iterasi, kegagalan pembangkitan tetangga, dan cache GA. Karena geometri serta kapasitas ikut berubah, ketiga dataset belum mengisolasi pengaruh jumlah paket. Perbedaan sesi medium dan batas pengukuran internal juga perlu diperhatikan.

GA memulai dari individu terbaik dalam populasi, sedangkan HC dan SA memakai satu state. Anggaran evaluasi yang berbeda membatasi perbandingan mekanisme pencarian. Seluruh 99 state awal dan akhir valid menurut kendala program.

Bab IV : Kesimpulan dan Saran

4.1 Kesimpulan

Program mengimplementasikan pemuatan satu truk melalui empat varian HC, SA, dan GA, dengan pemeriksaan dinding, tabrakan, tumpuan, fragile, serta kapasitas berat. Fungsi objektif alternatif tersedia, sedangkan seluruh eksperimen utama menggunakan jumlah nilai paket termuat.

Pengujian mencakup 99 run pada small, medium, dan large, masing-masing 33 run. Seluruh state awal dan akhir valid. Skor terbaik yang ditemukan berturut-turut adalah 445, 1.740, dan 4.320. Optimum global ketiga kasus belum ditetapkan oleh eksperimen ini.

Pada small, seluruh konfigurasi mencapai skor akhir 445. Hasil medium dan large memperlihatkan perbedaan kualitas, konsistensi, dan durasi yang perlu dinilai per dataset. Pada large, 100 generasi tidak memperbaiki hasil 50 generasi untuk populasi 30, dan populasi 60 menghasilkan rata-rata lebih rendah daripada populasi 30 pada 50 generasi. Tambahan populasi atau generasi perlu dinilai bersama biaya waktu dan hasil aktualnya.

4.2 Saran

Pengujian berikutnya dapat memperbanyak seed serta menyamakan anggaran evaluasi atau waktu antaralgoritma. Variasi dataset dengan jumlah paket yang sama tetapi geometri, kapasitas, dan proporsi fragile berbeda dapat membantu memisahkan pengaruh masing-masing karakteristik.

Pembangkitan tetangga dapat diperluas melalui anchor tambahan dan operator rotasi satu langkah 90 derajat. Metrik Sideways dan Random Restart dapat diperjelas agar percobaan, transisi diterima, iterasi dalam, dan titik awal dicatat terpisah. Pencarian eksak pada kasus kecil dapat digunakan sebagai pembanding kualitas solusi.

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM	: Fahd Muhammad Zahid/13524078 Sebagai Perwakilan
Fakultas/Sekolah	: STEI-K
Kode Mata Kuliah	: IF3170
Nama Mata Kuliah	: Intelegensi Artifisial
Judul Tugas	: Tugas Besar 1 AI Mejiro Express

Menyatakan bahwa Saya menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.
Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya	3
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	2
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	2
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	1
Lainnya, Jelaskan:	Digunakan untuk eksplorasi desain dan teori dasar yang digunakan	3

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 19 Oktober 2026



Fahd Muhammad Zahid
13524078

Lampiran

A. Pranala Github

https://github.com/alfarabi0642/IF3170_G01_Tandborste

B. Tabel Ceklis Spesifikasi

No	Poin	Ya	Tidak
1	Implementasi representasi state, fungsi objektif, dan constraints sesuai spesifikasi.	✓	
2	Implementasi algoritma Hill-Climbing.	✓	
3	Implementasi algoritma Simulated Annealing.	✓	
4	Implementasi algoritma Genetic Algorithm.	✓	
5	Visualisasi interaktif 3D untuk menampilkan initial state dan final state.	✓	
6	Visualisasi grafik dan metrik hasil pencarian setiap algoritma.	✓	
7	Pelaksanaan dan dokumentasi eksperimen sesuai ketentuan spesifikasi.	✓	
8	[Bonus] Implementasi fungsi objektif alternatif.	✓	
9	[Bonus] Implementasi seluruh varian Hill-Climbing.	✓	
10	[Bonus] Implementasi Multi-Truck.		✓

C. Tabel Pembagian Tugas

NIM	Pembagian Tugas
13524078	Implementasi algoritma Simulated Annealing, Genetic Algorithm, dan fungsi objektif alternatif. Pengujian, analisis hasil eksperimen, serta evaluasi pengaruh parameter pada kedua algoritma.
13524086	Implementasi representasi state, inisialisasi, fungsi objektif, dan penanganan constraints. Implementasi empat varian algoritma

	Hill-Climbing, pengujian, serta analisis dan perbandingan hasil eksperimen antaralgoritma.
13524094	Implementasi visualisasi 3D konfigurasi paket, visualisasi grafik metrik pencarian, serta integrasi visualisasi dengan hasil algoritma. Penyusunan deskripsi persoalan, kesimpulan dan saran, serta penggabungan dan finalisasi laporan.

Referensi

[1] GeeksforGeeks, "What is Simulated Annealing?," GeeksforGeeks - Artificial Intelligence, 2024. [Daring]. Tersedia: https://www.geeksforgeeks.org/artificial-intelligence/what-is-simulated-annealing/ [Diakses: 19-Okt-2026].

[2] S. Kirkpatrick, C. D. Gelatt, dan M. P. Vecchi, "Optimization by Simulated Annealing," Science, vol. 220, no. 4598, pp. 671–680, Mei 1983, doi: 10.1126/science.220.4598.671.

[3] GeeksforGeeks, "Genetic Algorithms," GeeksforGeeks - Data Structures and Algorithms, 2024. [Daring]. Tersedia: https://www.geeksforgeeks.org/dsa/genetic-algorithms/ [Diakses: 19-Okt-2026].

[4] J. H. Holland, Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence. Cambridge, MA: MIT Press, 1992.

[5] GeeksforGeeks, "Objective Function in Linear Programming," GeeksforGeeks - Mathematics, 2024. [Daring]. Tersedia: https://www.geeksforgeeks.org/maths/objective-function/ [Diakses: 19-Okt-2026].

[6] S. Russell dan P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ: Pearson, 2020.

[7] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=oGTa9kGE3P8. [Accessed: Sep. 28, 2026].

[8] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Sep. 28, 2026].

[9] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Sep. 28, 2026].

[10] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. Available: Beyond Classical Search: Simulated Annealing - YouTube.[Accessed: Sep. 28, 2026].

[11] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=aI41TGo9J9c. [Accessed: Sep. 28, 2026].